MARWADI UNIVERSITY

Department of Computer Science and Engineering – Artificial Intelligence and Machine Learning (CSE – AI/ML)

PROJECT REPORT

Design and Analysis of Algorithms (DAA)

Approximation Methodologies for the Travelling Salesman Problem: A Combinatorial Optimization Survey

A research-oriented survey with a validated experimental demonstration

Submitted by

Student name	Enrollment number
MARAPALLY RUUTHWIK	92400118870
ANNAVARAPU SRIVEK	92460118171

Under the guidance of

Project Guide: Mr. Sandeep Panchal

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project report entitled **"Approximation Methodologies for the Travelling Salesman Problem: A Combinatorial Optimization Survey"** is a record of the work carried out by **MARAPALLY RUUTHWIK** (Enrollment No. 92400118870) and **ANNAVARAPU SRIVEK** (Enrollment No. 92460118171) of the Department of Computer Science and Engineering – Artificial Intelligence and Machine Learning (CSE – AI/ML), **MARWADI UNIVERSITY**, during the academic year 2026–2027, under the guidance of **Mr. Sandeep Panchal**. The work is submitted for the Design and Analysis of Algorithms (DAA) project requirement.

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Mr. Sandeep Panchal**, for his guidance, encouragement and valuable suggestions throughout the preparation of this project on the Travelling Salesman Problem.

We thank the faculty of the Department of Computer Science and Engineering – Artificial Intelligence and Machine Learning (CSE – AI/ML), **MARWADI UNIVERSITY**, for providing the academic environment and resources needed to complete this work.

We are grateful to the authors of the textbooks and research papers cited in this report, whose work forms the basis of the survey. We also thank our families and friends for their constant support.

MARAPALLY RUUTHWIK
ANNAVARAPU SRIVEK

ABSTRACT

The Travelling Salesman Problem (TSP) asks for the cheapest closed tour that visits every city exactly once. It is NP-hard, and for general cost matrices no constant-factor approximation algorithm exists unless P = NP, so practical solution of large instances relies on approximation algorithms and heuristics. This report is a research-oriented survey of approximation methodologies for the TSP. It formulates the problem as a minimum-cost Hamiltonian cycle search, distinguishes symmetric, asymmetric and metric variants, and classifies solution approaches into exact algorithms, approximation algorithms with proven guarantees, constructive heuristics, local search, metaheuristics and learning-assisted methods. The double-tree (ratio 2) and Christofides (ratio 3/2) algorithms are analysed with proofs and with the exact assumptions (metric symmetric costs) under which their guarantees hold, and nearest neighbour, insertion heuristics, 2-opt, 3-opt, Lin–Kernighan-style search, simulated annealing, genetic algorithms, ant colony optimization and learning-based methods are reviewed from verified literature. Methods are compared by guarantee, solution quality, runtime, memory and implementation effort.

The survey is supported by a Python implementation whose every output tour is validated and whose cost is recomputed independently. Experiments on random Euclidean instances of up to 200 cities show, for example, that farthest insertion was the best constructive method, that 2-opt with Or-opt lowered the mean excess over the Held–Karp lower bound at 200 cities from 34.6 % (nearest neighbour) to 5.2 %, and that the proved bounds held on 300 metric test instances but were violated on random non-metric matrices. The report concludes that no single method is universally best, that a proven approximation ratio must not be confused with an observed gap, and that the choice of method depends on instance size, assumptions and the need for guarantees.

Keywords: Travelling Salesman Problem; approximation algorithms; combinatorial optimization; metric TSP; minimum spanning tree; Christofides' algorithm; local search; Lin–Kernighan; metaheuristics; Held–Karp.

TABLE OF CONTENTS

Chapter 1: Introduction **1**
1.1 Background of the Travelling Salesman Problem 1
1.2 History and Importance 1
1.3 Real-World Applications 1
1.4 Why Optimal Tours Are Hard and Why Approximation Helps 1
1.5 Motivation for the Survey 1
1.6 Problem Statement 2
1.7 Objectives 2
1.8 Scope and Limitations 2
1.9 Research Methodology 2
1.10 Expected Learning Outcomes 2
1.11 Organization of the Report 2
Chapter 2: Mathematical and Graph-Theoretic Foundations **3**
2.1 Basic Graph Theory 3
2.2 Cost Matrices and Distance Representations 3
2.3 Symmetric, Asymmetric and Metric TSP 3
2.4 Mathematical Formulation 4
2.5 Computational Complexity 4
2.6 Exact Algorithms, Approximation Algorithms and Heuristics 4
Chapter 3: Literature Review **5**
3.1 Early Exact Methods and Constructive Heuristics (1954–1963) 5
3.2 Lower Bounds and the Theory of Approximation (1970–1978) 5
3.3 Local Search and Variable-Depth Search (1973–2002) 5
3.4 Metaheuristics (1975–1997) 5
3.5 Modern Theory (1998–present) 5
3.6 Learning-Assisted Approaches (2015–present) 5
3.7 Comparison of Key Contributions 6
3.8 Research Gaps and Motivation for This Survey 7
Chapter 4: Classification of TSP Solution Approaches **8**
4.1 Exact Algorithms 8
4.2 Approximation Algorithms with Provable Guarantees 8
4.3 Constructive Heuristics 8
4.4 Local-Search and Improvement Methods 8
4.5 Metaheuristics 9
4.6 Learning-Assisted Approaches 9
4.7 Summary of the Classification 9
Chapter 5: Approximation Methodologies **10**
5.1 Nearest Neighbour 10
5.2 Cheapest Insertion and Other Constructive Heuristics 10
5.3 Minimum Spanning Tree-Based Approximation 11
5.4 Double-Tree Approximation 11
5.5 Christofides' Algorithm 12
5.6 Local Search: 2-opt, Or-opt and 3-opt 13
5.7 Lin–Kernighan-Style Methods 14
5.8 Metaheuristics 15
5.9 Learning-Based Methods 16
5.10 Method Selection 16
Chapter 6: Comparative Analysis **17**
6.1 Guarantees, Assumptions, Strengths and Limitations 17
6.2 Time, Memory and Practical Considerations 17
6.3 Guarantee Versus Observed Performance 18
6.4 No Method Is Universally Best 18
Chapter 7: Demonstration and Experimental Study **19**
7.1 Software Design 19
7.2 Input Format, Validation and How to Run 19
7.3 Experimental Environment and Settings 20

7.4 Exact Methods: Brute Force Versus Dynamic Programming....20
7.5 Worked Demonstration Instance....21
7.6 Solution Quality on Small Instances (Optimum Known)....22
7.7 Scaling Up to 200 Cities....23
7.8 Do the Guarantees Hold? A Metric Versus Non-Metric Test....24
7.9 Interpretation and Threats to Validity....25
Chapter 8: Applications, Limitations and Future Directions....26
8.1 Applications....26
8.2 Limitations of the Simplified TSP Model....26
8.3 Future Directions....26
Chapter 9: Conclusion....27
References....28
Appendix A: Additional Pseudocode....30
Appendix B: Validation Code....30
Appendix C: Sample Cost Matrices and Test Cases....31
Appendix D: Sample Program Outputs....31
Appendix E: Software Testing Checklist....32

LIST OF FIGURES

Figure 2.1: A five-city TSP instance. (a) The complete weighted graph K5. (b) The optimal…. 3
Figure 4.1: Classification of TSP solution approaches. Only the exact methods and the…. 8
Figure 5.1: Minimum spanning tree (blue, edge weights shown) of the 8-city demonstration…. 11
Figure 5.2: Double-tree algorithm on the demonstration instance. (a) The doubled MST (double…. 12
Figure 5.3: Workflow of Christofides' algorithm. The shaded boxes correspond to the six steps;…. 12
Figure 5.4: Christofides' algorithm on the demonstration instance. (a) The MST; shaded vertices…. 13
Figure 5.5: The 2-opt move. (a) Edges (a,b) and (c,d) of the tour cross. (b) Replacing them by…. 14
Figure 6.1: Largest tour-to-optimum ratio observed over 300 random metric instances (n = 9,…. 18
Figure 7.1: Module structure of tsp_survey.py (UML-style class/module diagram). Each box lists…. 19
Figure 7.2: Flowchart of the experimental procedure, including input validation and independent…. 19
Figure 7.3: Measured run time of the two exact methods against the number of cities (log scale)…. 21
Figure 7.4: The double-tree tour (cost 52, left) and the optimal tour (cost 42, right) on the…. 22
Figure 7.5: Mean gap to the exact optimum for each method on the small random instances of Table…. 22
Figure 7.6: Mean run time against number of cities for eight representative methods (logarithmic…. 24
Figure 7.7: Mean excess over the Held–Karp lower bound against number of cities…. 24

LIST OF TABLES

Table i.1: Abbreviations.... viii
Table i.2: Notation.... viii
Table 1.1: Organization of the report.... 2
Table 2.1: Variants of the TSP used in this report.... 3
Table 2.2: Three levels of solution quality.... 4
Table 3.1: Exact methods, bounds and approximation theory.... 6
Table 3.2: Practical heuristics, metaheuristics and learning-based methods.... 6
Table 4.1: Classification by the strength of the promise.... 9
Table 5.1: Summary of nearest neighbour.... 10
Table 5.2: Summary of insertion heuristics.... 10
Table 5.3: Summary of the double-tree algorithm.... 12
Table 5.4: Summary of Christofides' algorithm.... 13
Table 5.5: Summary of 2-opt and 3-opt.... 14
Table 5.6: Summary of Lin–Kernighan-style methods.... 14
Table 5.7: Summary of the metaheuristics.... 15
Table 5.8: Choosing a method.... 16
Table 6.1: Comparison of methods: category, assumptions, guarantee, strengths and limitations.... 17
Table 6.2: Time, memory, implementation effort and suitable sizes.... 17
Table 7.1: Experimental environment and settings.... 20
Table 7.2: Exact methods: candidates examined and mean run time (ms).... 20
Table 7.3: Demonstration instance: city coordinates and cost matrix (Manhattan distance).... 21
Table 7.4: Results on the 8-city instance (optimal cost 42). All tours validated; costs…. 21
Table 7.5: Gap to the exact optimum (%) on random Euclidean instances.... 22
Table 7.6: Mean run time (ms) against number of cities.... 23
Table 7.7: Mean excess over the Held–Karp lower bound (%).... 23
Table 7.8: Ratio to the exact optimum over 300 instances of 9 cities.... 24
Table C.1: Test cases.... 31
Table E.1: Software testing checklist.... 32

ABBREVIATIONS AND NOTATION

Table i.1: Abbreviations

Abbreviation	Meaning
TSP / STSP / ATSP	Travelling Salesman Problem / symmetric TSP / asymmetric TSP
DAA	Design and Analysis of Algorithms
MST	Minimum spanning tree
OPT	Cost of an optimal tour
NN	Nearest neighbour
SA / GA / ACO	Simulated annealing / genetic algorithm / ant colony optimization
LK / LKH	Lin–Kernighan heuristic / Helsgaun's implementation of it
PTAS	Polynomial-time approximation scheme
VRP	Vehicle-routing problem
LP / ILP	Linear programming / integer linear programming
LB	Lower bound (here: the Held–Karp 1-tree bound)

Table i.2: Notation

Symbol	Meaning
$G = (V, E)$	Graph with vertex set V (cities) and edge set E
n	Number of cities; cities are numbered $0,\ldots,n-1$
$D = [d_{ij}]$	Cost (distance) matrix; d_{ij} is the cost of going from city i to city j; $d_{ii} = 0$
π, $C(\pi)$	A tour (permutation of the cities) and its total cost
T, $w(T)$	Spanning tree and its total weight
O, M	Set of odd-degree vertices of T; a perfect matching on O
ρ	Approximation ratio (worst-case, over all instances of a class)
Δ	Change in tour cost caused by a move
Gap (%)	$100\cdot(\text{cost} - \text{OPT})/\text{OPT}$, measured on a particular instance

Chapter 1: Introduction

1.1 Background of the Travelling Salesman Problem

The **Travelling Salesman Problem (TSP)** asks a deceptively simple question: given a list of cities and the cost of travelling between every pair of them, what is the cheapest round trip that visits every city exactly once and returns to the starting city? In graph language, the cities are the vertices of a weighted graph, the travel costs are edge weights, and the task is to find a **minimum-cost Hamiltonian cycle** [1]. The problem is easy to state and easy to understand, yet it has become the standard test problem of combinatorial optimization: almost every important idea in the field, from linear-programming relaxations to local search and metaheuristics, has been tried on it first [2], [3].

The difficulty is the number of candidate solutions. With the starting city fixed, a symmetric instance with n cities has $(n-1)!/2$ distinct tours. For 10 cities this is 181,440 tours, for 15 cities about 4.4×10^{10}, and for 20 cities about 6.1×10^{16}. Checking all tours is therefore hopeless long before the instance size becomes interesting for real applications. Chapter 7 measures this growth directly (Table 7.2).

1.2 History and Importance

Dantzig, Fulkerson and Johnson showed in 1954 how linear programming with cutting planes could prove optimality for a 49-city instance, a landmark for the whole of integer programming [4]. Dynamic-programming algorithms that remove the factorial but remain exponential followed in 1962 [5], [6], and branch-and-bound algorithms were published in 1963 [7]. Karp's 1972 list of NP-complete problems included the Hamiltonian-cycle problem from which the hardness of the TSP follows [8], [1]. A broad account of the exact and heuristic methods developed up to the mid-1980s is given in [2], and the computational state of the art for exact solution is described in [3]; a readable narrative is in [9].

The TSP matters for three reasons. First, it is a **model problem**: methods invented for it (cutting planes, 1-tree bounds, k-opt local search, simulated annealing, ant colony optimization) have been reused for scheduling, routing, and design problems [2]. Second, it is a **benchmark**: standard instance libraries such as TSPLIB [10] allow different algorithms to be compared fairly. Third, it is a **theory testbed**: the search for the best possible approximation ratio for the TSP has produced a series of deep results that are still being improved [11], [12], [13].

1.3 Real-World Applications

Routes for delivery vehicles, the order in which a machine drills holes in a circuit board, the sequence in which a robot inspects points on a structure, and the order in which jobs are processed on a machine with sequence-dependent set-up times can all be modelled as TSP instances, or as close relatives of it [2], [14]. Chapter 8 discusses these applications and, equally important, the points at which the plain TSP model stops being a faithful description of the real problem.

1.4 Why Optimal Tours Are Hard and Why Approximation Helps

The decision version of the TSP (is there a tour of cost at most B?) is NP-complete, so no polynomial-time algorithm is known, and none exists unless P = NP [1]. The best known general exact algorithm in terms of worst-case time is the Held–Karp dynamic program, which needs $O(n^2\cdot2^n)$ time and $O(n\cdot2^n)$ memory [5], [6]. This removes the factorial but remains exponential; our measurements in Chapter 7 show it becoming slow for a pure-Python implementation at around 16 cities. Specialised exact solvers based on cutting planes and branch-and-cut handle much larger instances, but without a polynomial worst-case bound [3].

Approximation and heuristic methods trade guaranteed optimality for speed. Some of them come with a *proof* that the returned tour is never worse than a stated multiple of the optimum (for example, Christofides' algorithm for metric instances [15], [16]); others have no such proof but work very well in practice (for example, Lin–Kernighan-style local search [17], [18]). A student who wants to use these methods correctly must know which is which, and under which assumptions each guarantee holds. That distinction is the central theme of this report.

1.5 Motivation for the Survey

Textbook treatments usually present the 2-approximation and Christofides' 3/2-approximation for the *metric* TSP, and then move on. Several things are often left unsaid: that these guarantees disappear if the triangle inequality fails; that the general TSP cannot be approximated within any constant factor unless P = NP [19]; that popular practical heuristics (nearest neighbour, 2-opt, genetic algorithms) have no constant-factor guarantee; and that recent theory has finally improved the 3/2 barrier for metric instances [11] and obtained a constant-factor algorithm for the asymmetric case [12]. This project collects these threads into one consistent undergraduate-level survey and supports the discussion with a small, honest, reproducible experiment.

1.6 Problem Statement

Problem statement. To survey the approximation methodologies for the Travelling Salesman Problem; to state precisely the variant of the problem, the assumptions, and the guarantee (if any) of each method; to compare the methods on solution quality, running time, memory, and implementation effort; and to support the comparison with a validated, reproducible implementation, while keeping literature-derived claims separate from the project's own measurements.

1.7 Objectives

1. Formulate the TSP as a minimum-cost Hamiltonian cycle problem and explain symmetric, asymmetric and metric variants.
2. Classify solution approaches into exact algorithms, approximation algorithms with proven guarantees, practical heuristics, metaheuristics and learning-assisted methods.
3. Explain, with pseudocode and proofs where appropriate, the double-tree (MST) and Christofides algorithms, and summarise local-search, metaheuristic and learning-based methods.
4. Implement the main methods in Python, validate every returned tour, and compare them on identical inputs against brute force and dynamic programming for small instances.
5. Distinguish clearly between a proven approximation ratio and an observed gap on test instances.

1.8 Scope and Limitations

The report covers the classical (single-vehicle) TSP in its symmetric, metric and, briefly, asymmetric forms. Vehicle-routing and other constrained variants are mentioned only as extensions. The survey is limited to well-established sources: textbooks, foundational papers and a small number of recent landmark papers. It is not an exhaustive bibliography. The experimental study uses a pure-Python implementation on a modest two-core cloud machine and instance sizes up to 200 cities; it is a demonstration of method behaviour, **not a benchmark** of state-of-the-art solvers. No neural method was implemented; learning-based approaches are discussed from the literature only.

1.9 Research Methodology

The project followed a four-step methodology. (1) *Literature study*: foundational papers and textbooks were identified, and bibliographic details were checked against online records wherever possible. (2) *Analysis*: each algorithm was analysed for its idea, steps, complexity, assumptions and guarantee. (3) *Implementation*: the algorithms were implemented in one Python file, with automated self-tests. (4) *Experiment*: the programme was executed on documented instances, and every number in Chapter 7 was copied from the programme's output file `results.json` into the report by script, so that no measurement is typed by hand.

1.10 Expected Learning Outcomes

After studying this report a reader should be able to: define the TSP and Hamiltonian cycles; explain why the TSP is NP-hard; run and trace the double-tree and Christofides algorithms by hand; prove the 2 and 3/2 bounds for metric instances; explain why these bounds fail for arbitrary cost matrices; implement 2-opt; and choose an appropriate method for a given instance size and accuracy requirement.

1.11 Organization of the Report

Table 1.1: Organization of the report

Chapter	Content
1 Introduction	Background, motivation, problem statement, objectives, scope, methodology.
2 Mathematical Foundations	Graphs, cost matrices, TSP variants, formulation, complexity, exact vs. approximate vs. heuristic.
3 Literature Review	Development of TSP methods; comparison table; research gaps.
4 Classification	Exact, approximation, constructive, local search, metaheuristic and learning-assisted methods.
5 Approximation Methodologies	Detailed treatment of ten method families, with pseudocode, complexity and guarantees.
6 Comparative Analysis	Comparison tables; guarantee versus observed performance.
7 Demonstration and Experiments	Software design, validation, and measured results.
8 Applications and Future Directions	Applications, model limitations, research directions.
9 Conclusion	Summary of supported conclusions and limitations.
Appendices	Pseudocode, validation code, sample data and outputs, testing checklist.

Chapter 2: Mathematical and Graph-Theoretic Foundations

2.1 Basic Graph Theory

A **graph** $G = (V, E)$ consists of a set V of **vertices** (here, cities) and a set E of **edges** (here, direct connections). A **weighted graph** assigns a number $w(e) \geq 0$, the cost or distance, to every edge. A **path** is a sequence of distinct vertices in which consecutive vertices are joined by edges; a **cycle** is a path that returns to its starting vertex. A **Hamiltonian cycle** is a cycle that visits *every* vertex exactly once. The **complete graph** K_n has an edge between every pair of its n vertices; the TSP is normally posed on a complete graph, so that a Hamiltonian cycle always exists. If an application provides only some roads, the missing pairs are given the length of the shortest path between them.

A **tree** is a connected graph without cycles; a **spanning tree** of G is a tree containing all vertices of G; and a **minimum spanning tree (MST)** is a spanning tree of minimum total weight. A graph is **Eulerian** if it has a closed walk that uses every edge exactly once; a connected multigraph is Eulerian exactly when every vertex has even degree. These two facts, together with the *handshake lemma* (the number of odd-degree vertices in any graph is even), are the building blocks of the MST-based algorithms of Chapter 5.

2.2 Cost Matrices and Distance Representations

The input of a TSP instance with cities $0,\ldots,n-1$ is an $n \times n$ **cost matrix** $D = [d_{ij}]$, where d_{ij} is the cost of travelling from city i to city j and $d_{ii} = 0$. When the cities are points in the plane, D is computed from coordinates, for instance with the Euclidean distance $d_{ij} = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$ or the Manhattan distance $|x_i - x_j| + |y_i - y_j|$. Figure 2.1 shows a five-city instance as a weighted complete graph and the optimal tour found by exhaustive search; its cost matrix is the one listed in Appendix C. This particular matrix was chosen for illustration and is deliberately **not** metric (for example, $d(\text{A,D}) = 19$ although $\text{A} \to \text{E} \to \text{D}$ costs $8 + 4 = 12$).

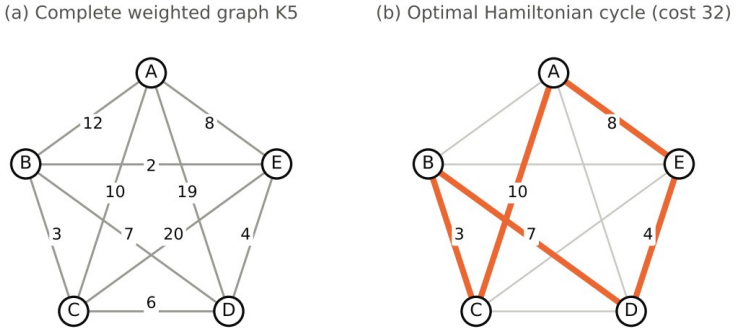


Figure 2.1: A five-city TSP instance. (a) The complete weighted graph K5. (b) The optimal Hamiltonian cycle A–E–D–B–C–A of cost 32, found by exhaustive search. The instance is illustrative and not metric.

2.3 Symmetric, Asymmetric and Metric TSP

Table 2.1: Variants of the TSP used in this report

Variant	Definition	Why it matters
Symmetric TSP (STSP)	$d_{ij} = d_{ji}$ for all i, j. Edges are undirected.	Most classical heuristics and bounds (1-tree, 2-opt, Christofides) assume symmetry.
Asymmetric TSP (ATSP)	$d_{ij} \neq d_{ji}$ is allowed (one-way streets, directed set-up times).	MST- and 2-opt-based methods do not apply directly; special algorithms are needed [12].
Metric TSP	Symmetric and the **triangle inequality** holds: $d_{ij} \leq d_{ik} + d_{kj}$ for all i, j, k.	Allows "shortcutting" without increasing cost; this is the setting of the 2- and 3/2-approximation guarantees.
Euclidean TSP	Cities are points in the plane (or $\mathbb{R}^d$) and d_{ij} is their Euclidean distance. Every Euclidean instance is metric.	Admits a polynomial-time approximation scheme in fixed dimension [20].

Every Euclidean instance is metric, every metric instance is symmetric, but not conversely. **A guarantee proved for one class must not be applied to a larger class.** This report always states the class in which a guarantee holds, and Section 7.8 demonstrates experimentally what happens when the triangle inequality is violated.

2.4 Mathematical Formulation

2.4.1 Permutation form

A tour can be written as a permutation π of the cities, where π(*k*) is the city visited in position *k*. The **objective** is

$$\min_{\pi} \quad C(\pi) = \sum_{k=1}^{n-1} d_{\pi(k),\pi(k+1)} + d_{\pi(n),\pi(1)} \tag{2.1}$$

where the last term closes the cycle by returning to the starting city. Let OPT denote the minimum value of $C(\pi)$ over all $(n-1)!$ permutations with $\pi(1)$ fixed.

2.4.2 Integer-programming form (Dantzig–Fulkerson–Johnson)

Let the binary variable x_{ij} equal 1 if the tour travels directly from city *i* to city *j*. For the asymmetric case the formulation is

$$\min \quad \sum_{i} \sum_{j \neq i} d_{ij}\, x_{ij} \tag{2.2}$$

$$\sum_{j \neq i} x_{ij} = 1 \quad \text{for every city } i \qquad \text{(leave each city exactly once)} \tag{2.3}$$

$$\sum_{i \neq j} x_{ij} = 1 \quad \text{for every city } j \qquad \text{(enter each city exactly once)} \tag{2.4}$$

$$\sum_{i \in S} \sum_{j \in S,\, j \neq i} x_{ij} \leq |S| - 1 \quad \text{for every subset } S,\ 2 \leq |S| \leq n-1 \tag{2.5}$$

$$x_{ij} \in \{0, 1\} \tag{2.6}$$

The two **degree constraints** make every city have exactly one outgoing and one incoming edge, so the chosen edges form a collection of disjoint directed cycles. They do not, however, force a *single* cycle: a solution may consist of two or more separate small cycles called **subtours**. The **subtour-elimination constraints** (the fourth equation) forbid this: for any proper subset *S* of cities, at most $|S|-1$ chosen edges may lie inside *S*, so a cycle cannot close within *S* alone [4]. There are exponentially many such constraints, which is why cutting-plane methods add them only when they are violated. Dropping integrality gives the LP relaxation whose optimum is a lower bound on OPT; the Held–Karp 1-tree bound used in Chapter 7 is a closely related lower bound [21], [22].

2.5 Computational Complexity

- **Hamiltonian cycle** is NP-complete [8]; deciding whether a TSP instance has a tour of cost ≤ *B* is therefore NP-complete, and the optimization version is NP-hard [1].
- **General TSP cannot be approximated** within *any* constant factor ρ in polynomial time unless P = NP [19]. The reason is that a hypothetical ρ-approximation could be used to decide the Hamiltonian-cycle problem by assigning cost 1 to graph edges and a huge cost to non-edges.
- **Metric TSP is APX-hard**: a sufficiently small constant improvement over the optimum is impossible unless P = NP. The best published inapproximability constant that we verified is 123/122 for the symmetric case and 75/74 for the asymmetric case [23]. The gap between 123/122 ≈ 1.008 and the best known upper bound (slightly below 3/2, Chapter 3) is still large.
- **Euclidean TSP** has a polynomial-time approximation scheme for fixed dimension [20]; its running time grows rapidly as the desired accuracy increases, so it is of theoretical rather than everyday practical value.

2.6 Exact Algorithms, Approximation Algorithms and Heuristics

Three different kinds of statement can be made about a tour, and the report keeps them apart.

Table 2.2: Three levels of solution quality

Level	What is claimed	How it is established	Example
Optimal tour	$C(\pi) = \text{OPT}$.	Exhaustive search, dynamic programming, branch-and-bound or cutting planes; worst-case exponential time.	Held–Karp DP
Proven approximation guarantee	$C(\pi) \leq \rho \cdot \text{OPT}$ for **every** instance in a stated class, with polynomial running time.	A mathematical proof under explicit assumptions (class of instances, e.g. metric symmetric).	Double-tree (ρ = 2), Christofides (ρ = 3/2)
Good tour empirically	On the tested instances, the tour is on average *x* % above the optimum or a lower bound.	Experiments. Says **nothing** about the worst case or about untested instances.	2-opt, Lin–Kernighan, simulated annealing

An **approximation algorithm** with ratio $\rho \geq 1$ for a minimization problem is a polynomial-time algorithm *A* such that $A(I) \leq \rho \cdot \text{OPT}(I)$ for every instance *I* in the stated class [24], [25]. The ratio is therefore a *supremum over all instances*; it is a property of the algorithm and the instance class, not of any particular dataset. A **heuristic** is any method without such a proof. Heuristic does not mean bad: the best practical TSP solvers are heuristics, but their quality is documented by experiments, not by theorems [26].

Chapter 3: Literature Review

This chapter follows the development of TSP solution methods from the 1950s to the present. Every source was selected because it is a foundational paper, an established textbook or a recent landmark result.

3.1 Early Exact Methods and Constructive Heuristics (1954–1963)

The first successful solution of a non-trivial TSP instance came from Dantzig, Fulkerson and Johnson [4], who solved a 49-city problem by adding subtour-elimination constraints to a linear program one at a time, exactly as described in Section 2.4. Croes proposed an *inversion* method that repeatedly reverses a section of the tour when this shortens it [27]; this is the idea now called 2-opt. Held and Karp [5] and, independently, Bellman [6] gave the dynamic program that solves the TSP in $O(n^2 2^n)$ time, and Little, Murty, Sweeney and Karel [7] introduced branch-and-bound with a lower bound obtained from a reduced cost matrix. All of these methods find optimal tours but have exponential worst-case running time; in particular, they cannot be called approximation algorithms.

3.2 Lower Bounds and the Theory of Approximation (1970–1978)

Held and Karp showed in 1970 how to compute a strong lower bound from *1-trees*, a spanning tree on all cities but one plus two edges at the remaining city, together with node penalties adjusted by subgradient optimization [21], [22]. This bound is used in the experiments of Chapter 7 to judge tour quality on instances too large for exact solution. In 1976 Sahni and Gonzalez proved that, for the general TSP, no polynomial-time algorithm can guarantee any constant ratio unless P = NP [19]. This makes the triangle inequality essential for any positive result. Rosenkrantz, Stearns and Lewis then analysed several heuristics in the metric setting: nearest neighbour has a ratio that grows at most logarithmically with n, while the nearest- and cheapest-insertion heuristics and the doubled-MST algorithm have ratio 2 [28]. Christofides [15] improved the ratio to 3/2 by adding a minimum-weight perfect matching on the odd-degree vertices of the MST; the same bound was obtained independently in the Soviet literature by Serdyukov [16]. For more than four decades this remained the best known guarantee for metric TSP.

3.3 Local Search and Variable-Depth Search (1973–2002)

Lin and Kernighan's 1973 heuristic generalised 2-opt and 3-opt into a *variable-depth* search that builds a sequence of edge exchanges as long as the cumulative gain remains positive [17]. Helsgaun's 2000 implementation (LKH) strengthened it with 5-opt basic moves and candidate sets chosen by an "α-nearness" measure derived from 1-trees, and reports very high solution quality on benchmark instances [18]. Chandra, Karloff and Tovey studied worst-case behaviour of k-opt local search and showed that local optima can be far from the optimum, which explains why local-search methods carry no constant-factor guarantee [29]. The extensive experimental study of Johnson and McGeoch compares tour construction, local search and Lin–Kernighan variants on random and library instances and remains the standard reference for the empirical ranking of these methods [26].

3.4 Metaheuristics (1975–1997)

Metaheuristics are general-purpose search frameworks that were applied to the TSP early. Kirkpatrick, Gelatt and Vecchi introduced *simulated annealing*, which accepts uphill moves with a temperature-dependent probability so that the search can escape local minima [30]. Genetic algorithms, popularised by Holland and Goldberg, maintain a population of tours and recombine them [31], [32]. Dorigo, Maniezzo and Colorni's *Ant System* builds tours probabilistically from pheromone trails reinforced by good tours [33], and the *Ant Colony System* refined this with a local pheromone update and a more greedy transition rule [34]. None of these methods has a worst-case approximation guarantee for the TSP; their value is their flexibility and their typically good empirical performance when carefully tuned.

3.5 Modern Theory (1998–present)

Arora showed in 1998 that the Euclidean TSP in fixed dimension has a polynomial-time approximation scheme: for every $\varepsilon > 0$ a tour of cost at most $(1+\varepsilon)\cdot\mathrm{OPT}$ can be found in polynomial time [20]. Karpinski, Lampis and Schmied proved the inapproximability bounds 123/122 (symmetric) and 75/74 (asymmetric) [23]. In 2021 Karlin, Klein and Oveis Gharan gave a *randomized* algorithm with ratio $3/2 - \varepsilon$ for metric TSP, for some constant $\varepsilon > 10^{-36}$, the first improvement over Christofides' ratio for general metrics [11]. The improvement is tiny numerically but breaks a barrier that stood for over 40 years. For the asymmetric TSP, Svensson, Tarnawski and Végh obtained the first constant-factor approximation algorithm [12]. The monograph of Traub and Vygen gives a unified treatment of these developments [13], and the algorithm textbooks [35], [24], [25] cover the classical 2 and 3/2 results.

3.6 Learning-Assisted Approaches (2015–present)

Vinyals, Fortunato and Jaitly introduced *pointer networks*, neural sequence models whose output is a pointer to an input element; they applied them to the TSP among other problems [36]. Bello et al. trained such networks with reinforcement learning, so that no optimal tours are needed as labels [37], and Kool, van Hoof and Welling proposed an attention-based model trained with a REINFORCE baseline for TSP and routing variants [38]. Bengio,

Lodi and Prouvost survey how machine learning can be combined with combinatorial-optimization solvers [39]. These methods produce a tour quickly once trained, but they provide **no worst-case approximation guarantee**, and generalisation to larger or differently distributed instances must be demonstrated experimentally.

3.7 Comparison of Key Contributions

Tables 3.1 and 3.2 summarise the main sources. "Guarantee" lists a proven worst-case statement only; "none" means that the cited work does not claim one.

Table 3.1: Exact methods, bounds and approximation theory

Reference, year	Main approach	Main contribution	Assumptions	Guarantee	Strengths	Limitations
[4], 1954	LP with cutting planes	Subtour-elimination formulation; 49-city solution	Any cost matrix (symmetric form used)	Exact (optimal)	Optimality proof; basis of modern solvers	Exponential worst case; needs LP solver
[5], [6], 1962	Dynamic programming over subsets	$O(n^2 2^n)$ exact algorithm	Any cost matrix	Exact (optimal)	Simple; deterministic; first sub-factorial bound	$O(n2^n)$ memory; limited to small n
[7], 1963	Branch and bound	Reduced-matrix lower bound and tree search	Any cost matrix	Exact (optimal)	Often far faster than enumeration	Worst case exponential; depends on bound quality
[21], [22], 1970–71	1-tree relaxation with node penalties	Strong lower bound on OPT	Symmetric costs	Lower bound (not a tour)	Typically within about 1 % of OPT on random instances [26]	Does not construct a tour by itself
[19], 1976	Complexity reduction	No constant ratio for general TSP	Arbitrary (non-metric) costs	Inapproximability unless P = NP	Shows why metric assumption is needed	Negative result only
[28], 1977	Analysis of NN, insertion, doubled MST	Ratios: NN logarithmic in n; insertion 2; double-tree 2	Metric symmetric	See main approach	First systematic worst-case study	Ratios for some heuristics are far above practice
[15], [16], 1976 / 1978	MST + perfect matching on odd vertices	3/2-approximation	Metric symmetric	$3/2 \cdot OPT$	Simple proof; best for 40+ years	Needs matching ($O(n^3)$); practice often better than bound
[20], 1998	Randomised geometric dissection + DP	PTAS for Euclidean TSP in fixed dimension	Euclidean, fixed dimension	$(1+\varepsilon)\cdot OPT$ for every $\varepsilon > 0$	Arbitrary accuracy in theory	Running time grows quickly as ε shrinks
[12], 2020	Reduction to subtour partition cover	Constant-factor ATSP approximation	Asymmetric costs with triangle inequality	Constant factor (large constant)	First constant for ATSP	Theoretical interest; constant is large
[11], 2021	Randomised tree sampling (strongly Rayleigh)	$(3/2 - \varepsilon)$ for metric TSP, $\varepsilon > 10^{-36}$	Metric symmetric	$(3/2 - \varepsilon)\cdot OPT$, randomised	Breaks the 3/2 barrier	ε is astronomically small; no practical gain

Table 3.2: Practical heuristics, metaheuristics and learning-based methods

Reference, year	Main approach	Main contribution	Assumptions	Guarantee	Strengths	Limitations
[27], 1958	Segment reversal (inversion)	Origin of 2-opt	Symmetric costs	None	Simple, fast, effective	Stops at local optimum
[17], 1973	Variable-depth edge exchange	Lin–Kernighan heuristic	Symmetric costs	None	Strong quality for modest time	Delicate to implement; no worst-case bound
[29], 1999	Worst-case analysis of k-opt	Local optima can be far from OPT	Metric and Euclidean cases	Shows ratio grows with n in the worst case	Explains the limits of local search	Analysis of worst case, not typical case
[30], 1983	Simulated annealing	Probabilistic acceptance with cooling	None specific	None	General; escapes local minima	Parameter tuning; no guarantee
[33], [34], 1996–97	Ant colony optimization	Pheromone-guided construction	None specific	None	Parallel, adaptive	Slower; sensitive to parameters
[31], [32], 1975/89	Genetic algorithms	Population-based search with crossover	None specific	None	Flexible representation	Needs good operators; poor scaling if plain
[18], 2000	Lin–Kernighan with 5-opt moves and α-nearness	LKH implementation	Symmetric costs	None	Very high reported quality	Complex; preprocessing cost
[26], 2002	Experimental comparison	Empirical ranking of heuristics	Random and library instances	Not applicable	Large, careful study	Instance-dependent conclusions
[36], [37], [38], 2015–19	Neural sequence/attention models, RL	Learned construction policies for TSP	Training instances from a distribution	None	Fast inference after training	Generalisation and optimality not guaranteed
[39], 2021	Survey of ML for CO	Taxonomy of learning + solver combinations	Not applicable	Not applicable	Organises the field	Survey; not a TSP benchmark

3.8 Research Gaps and Motivation for This Survey

- **Theory–practice gap.** The best proven ratio for metric TSP is barely below 3/2 [11], yet practical heuristics routinely produce tours within a few percent of optimal [26]. The two kinds of statement are easily confused.
- **Assumption blindness.** Guarantees for the double-tree and Christofides algorithms hold for metric symmetric instances; real cost data (road networks with one-way streets, time-dependent travel times) often violate these assumptions [19], [12].
- **Guarantee-free methods dominate practice.** Local search, metaheuristics and learning-based methods have no worst-case guarantee; their evaluation is empirical and must be reported with honest limitations [29], [39].
- **Reproducibility.** Many teaching resources show an algorithm without a validated implementation. This survey supplies a small, self-testing implementation with documented instances.

These gaps motivate the survey: to present the approximation landscape with its assumptions made explicit, and to demonstrate on a small scale how guarantees and observed behaviour relate.

Chapter 4: Classification of TSP Solution Approaches

Solution approaches differ in what they promise, and this classification is based on that promise. Figure 4.1 shows the families discussed in this chapter and indicates which of them carry a proven worst-case guarantee.

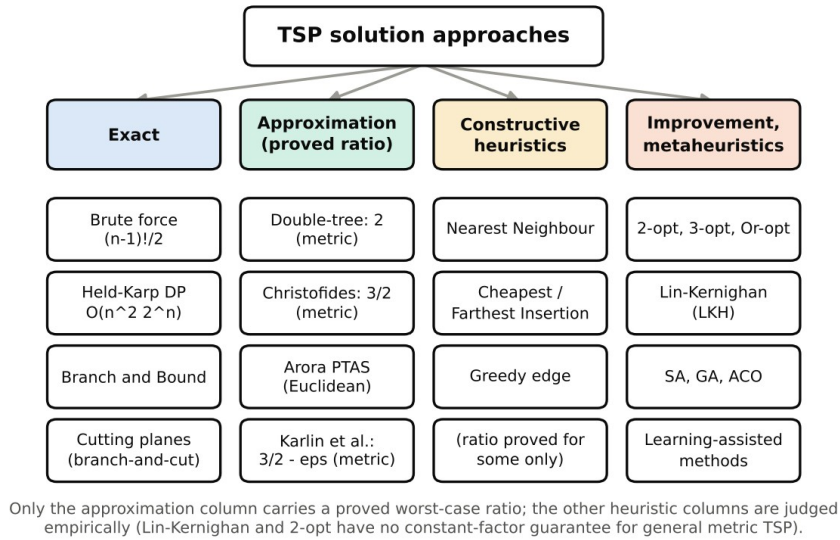


Figure 4.1: Classification of TSP solution approaches. Only the exact methods and the approximation algorithms (with stated assumptions) carry proofs: optimality, or a bounded ratio.

4.1 Exact Algorithms

Brute-force enumeration generates every tour, computes its cost and keeps the cheapest: $(n-1)!/2$ tours for symmetric instances. It is the simplest correct method and serves as the reference solution in Chapter 7. **Dynamic programming (Held–Karp)** exploits the fact that the best way to finish a tour depends only on the *set* of visited cities and the current city, not on the order in which the set was visited: with $f(S, j)$ the cheapest path that starts at city 0, visits exactly the cities in S and ends at j, it satisfies $f(S, j) = \min_{i \in S \setminus \{j\}} [f(S \setminus \{j\}, i) + d_{ij}]$, giving $O(n^2 2^n)$ time and $O(n2^n)$ space [5], [6]. **Branch and bound** searches a tree of partial tours and discards a branch as soon as a lower bound on its best completion exceeds the cost of the best complete tour found so far [7]; bounds can come from reduced cost matrices, assignment relaxations or 1-trees [21]. Its worst case is exponential, but good bounds prune most of the tree. **Cutting-plane and branch-and-cut** methods, which solve the integer program of Section 2.4 using LP relaxations and violated subtour constraints, are the basis of the best exact solvers [4], [3]. Branch and bound is thus an exact method in this report, discussed for context; the focus of the survey is the approximation side.

4.2 Approximation Algorithms with Provable Guarantees

These algorithms run in polynomial time and carry a *proof*, for a specified instance class, that the tour is within a factor ρ of the optimum. The **double-tree** algorithm ($\rho = 2$) and **Christofides' algorithm** ($\rho = 3/2$) are the classical examples for metric symmetric TSP [28], [15], [16]. Newer results include a randomised $(3/2 - \varepsilon)$-approximation for metric TSP [11], a PTAS for Euclidean TSP in fixed dimension [20] and a constant-factor algorithm for ATSP [12]. The insertion heuristics nearest and cheapest insertion also have a proven ratio of 2 in the metric case [28]; nearest neighbour has only a logarithmic bound [28].

4.3 Constructive Heuristics

Constructive heuristics build one tour from scratch. **Nearest neighbour** repeatedly moves to the closest unvisited city. **Cheapest insertion** and **farthest insertion** maintain a partial tour and insert one new city at a time at the position that increases the cost least. They are very fast and are mostly used to produce a starting tour for improvement methods; their guarantees, where they exist, are weaker than those of Christofides' algorithm even though their observed tours are often better (Chapter 7).

4.4 Local-Search and Improvement Methods

A local-search method starts from a tour and repeatedly replaces it by a better tour in its *neighbourhood* until none exists. **2-opt** replaces two edges by two others, **3-opt** replaces three, and **Lin–Kernighan** methods chain exchanges of variable depth [27], [17]. They finish in a local optimum, whose quality can be arbitrarily poor in the worst case [29] yet is usually excellent in practice [26].

4.5 Metaheuristics

Simulated annealing, genetic algorithms and ant colony optimization are general search frameworks with randomisation and a memory of previous search [30], [32], [33]. They can be adapted to many problems, but they provide neither optimality nor a worst-case ratio for the TSP, and their results depend on parameter choices and random seeds.

4.6 Learning-Assisted Approaches

Learning-assisted methods use data to guide the search: either an end-to-end neural model that outputs a tour [36], [37], [38], or a hybrid in which a learned component helps a classical solver (for example by choosing branching or candidate edges) [39]. They are *heuristics* from the viewpoint of guarantees.

4.7 Summary of the Classification

Table 4.1: Classification by the strength of the promise

Class	Representative methods	Promise	Typical use
Exact	Brute force, Held–Karp DP, branch and bound, branch and cut	Optimal tour (possibly exponential time)	Small instances; benchmarks; when optimality is mandatory
Approximation (proved ratio)	Double-tree (2), Christofides (3/2), randomised (3/2−ϵ), Euclidean PTAS	Tour ≤ ρ·OPT for every instance of a stated class, in polynomial time	Settings that need a worst-case guarantee; theory
Constructive heuristic	Nearest neighbour, cheapest and farthest insertion	Fast tour; ratio known only for some (metric)	Starting tours; very large instances
Local search	2-opt, 3-opt, Or-opt, Lin–Kernighan	Local optimum; no constant ratio	Default practical improvement step
Metaheuristic	Simulated annealing, genetic algorithm, ant colony	None	Flexible search when extra constraints exist
Learning-assisted	Pointer networks, attention models, learned guidance	None	Repeated solving of similar instances; research

Important. Only the second row of Table 4.1 consists of *approximation algorithms* in the strict sense. Calling 2-opt, a genetic algorithm or a neural network an "approximation algorithm with ratio ρ" would be incorrect.

Chapter 5: Approximation Methodologies

This is the central chapter of the survey. Each method is described by its idea, input and output, steps, pseudocode, complexity, strengths, limitations, applications, guarantee and the assumptions behind that guarantee. Throughout, D is the $n \times n$ cost matrix, cities are $0,\ldots,n-1$, a tour is a list of all cities and the closing edge from the last city back to the first is implicit. "Time" refers to the number of elementary operations, and "space" refers to memory beyond the input matrix, unless stated otherwise.

5.1 Nearest Neighbour

Concept and motivation. Nearest neighbour (NN) is the most natural greedy idea: a salesman who always travels to the closest city not yet visited. It is the simplest tour-construction heuristic and is used as a quick baseline and as a starting tour for improvement methods.

Steps. (1) Choose a start city and mark it visited. (2) Among the unvisited cities, choose the one nearest to the current city and move there. (3) Repeat until every city has been visited. (4) Return to the start city.

Algorithm 5.1 NEAREST-NEIGHBOUR(D, s)

```
tour ← [s];  U ← V \ {s};  cur ← s
while U ≠ ∅ do
   next ← the city j ∈ U with the smallest D[cur][j]
   append next to tour;  remove next from U;  cur ← next
return tour          (the edge from the last city back to s closes the cycle)
```

Table 5.1: Summary of nearest neighbour

Item	Detail
Input / output	Cost matrix D and a start city s; a Hamiltonian cycle.
Time / space	$O(n^2)$ time (each of n steps scans up to n cities); $O(n)$ extra space.
Guarantee	No constant-factor guarantee. For **metric symmetric** instances the tour cost is at most $\frac{1}{2}(\lceil \log_2 n \rceil + 1)\cdot$OPT, and there are instances on which the ratio grows logarithmically with n [28]. For non-metric matrices no guarantee is claimed.
Assumptions	Metric symmetric TSP for the logarithmic bound above; none for the algorithm to run.
Strengths	Very simple and fast; a valid tour for any cost matrix, even asymmetric.
Limitations	Short-sighted: early choices force very long edges at the end. Observed gaps of 7–35 % in Chapter 7.
Applications	Initial tours for local search; quick estimates; very large instances where time dominates.

5.2 Cheapest Insertion and Other Constructive Heuristics

Concept. Insertion heuristics keep a *partial tour*, that is a cycle through some of the cities, and enlarge it one city at a time. The **insertion cost** of city k between consecutive tour cities i and j is the increase in tour length

$$\Delta(i,\ k,\ j) = d_{ik} + d_{kj} - d_{ij} \tag{5.1}$$

Cheapest insertion examines every uninserted city and every tour edge and performs the (city, edge) pair with the smallest Δ. **Farthest insertion** first selects the uninserted city whose distance to the current tour is *largest* and then inserts it at its cheapest position; the idea is to fix the overall shape of the tour first and fill in details later. Route quality therefore depends on *which* city is inserted next as much as on *where*; this is why farthest insertion, which seems counter-intuitive, performs best among the constructive methods in our experiments (Section 7.7). Other classical constructive heuristics, such as greedy edge matching and the savings method, are compared experimentally in [26].

Algorithm 5.2 CHEAPEST-INSERTION(D) (farthest-insertion variant in brackets)

```
s ← the city ≠ 0 with the smallest D[0][s]     [s ← the city ≠ 0 with the largest D[0]
[s]]
tour ← [0, s];  U ← V \ {0, s}
while U ≠ ∅ do
   (k, e=(i,j)) ← the city k ∈ U and tour edge (i, j) minimising D[i][k] + D[k][j] −
   D[i][j]
   [k ← the city in U maximising min over tour cities v of D[k][v];
    (i, j) ← the tour edge minimising D[i][k] + D[k][j] − D[i][j]]
   insert k between i and j in tour;  remove k from U
return tour
```

Table 5.2: Summary of insertion heuristics

Item	Detail
Input / output	Cost matrix D; a Hamiltonian cycle.

Item	Detail
Time / space	The direct implementation of Chapter 7 is O(n^3) time and O(n) space; with priority queues cheapest insertion can be made O(n^2 log n) [26].
Guarantee	Cheapest insertion (and nearest insertion) have ratio at most 2 for **metric symmetric** instances [28]. No such bound is claimed here for farthest insertion.
Assumptions	Metric symmetric TSP for the ratio-2 bound. The heuristic itself needs no assumption.
Strengths	Better tours than nearest neighbour; easily adapted to side constraints such as time windows.
Limitations	Slower than NN; greedy choices cannot be undone; observed quality depends on the insertion rule.
Applications	Vehicle routing construction heuristics; starting tours for improvement [14].

5.3 Minimum Spanning Tree-Based Approximation

Concept and motivation. A minimum spanning tree connects all cities with the least total edge weight but is not a tour. The key observation is that the MST is a *lower bound* on the optimal tour:

Lemma 5.1. For any symmetric instance, w(MST) ≤ OPT.

Proof. Delete any one edge from an optimal tour. What remains is a path through all cities, which is a spanning tree, and its weight is at most OPT because weights are non-negative. The MST has no greater weight than this spanning tree. ∎

Since the MST is cheap to compute, it is a good starting point: Prim's algorithm builds it in O(n^2) time on a dense cost matrix [40], [35]. The remaining task is to convert a tree into a tour without paying much more. The **triangle inequality is what makes this conversion cheap**: if $d_{ij} \le d_{ik} + d_{kj}$ for all triples, then skipping an already visited city (a "shortcut") never increases the length of a walk. Without the triangle inequality a shortcut can be arbitrarily expensive, and the bounds of Sections 5.4 and 5.5 fail (demonstrated in Section 7.8).

Algorithm 5.3 PRIM-MST(D)

```
key[v] ← ∞ for all v;  key[0] ← 0;  parent[v] ← none;  inTree[v] ← false
repeat n times
   u ← the city with inTree[u] = false and the smallest key[u];  inTree[u] ← true
   if parent[u] exists then output edge (parent[u], u)
   for each v with inTree[v] = false and D[u][v] < key[v] do
      key[v] ← D[u][v];  parent[v] ← u
```

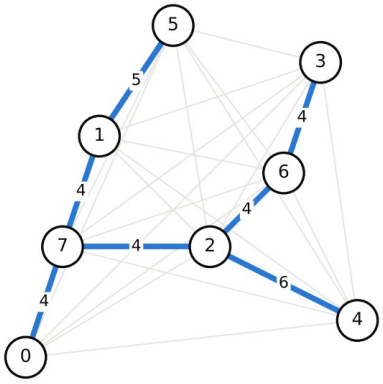


Figure 5.1: Minimum spanning tree (blue, edge weights shown) of the 8-city demonstration instance of Section 7.5. Grey lines are the remaining edges of the complete graph. The tree has weight 31, which is below the optimal tour cost of 42, as Lemma 5.1 requires.

5.4 Double-Tree Approximation

Concept. Double every edge of the MST. The resulting multigraph has all degrees even, so it contains an Eulerian circuit that traverses every doubled edge once; this circuit visits all cities, but some of them several times. Replacing every repeated visit by a shortcut yields a Hamiltonian cycle.

Steps. (1) Compute an MST T. (2) Double its edges to obtain a multigraph T^2. (3) Find an Eulerian circuit of T^2. (4) Walk along the circuit, keeping the first occurrence of every city and skipping repeats. Following the circuit of the doubled tree is the same as a *preorder traversal* of the tree, so the implementation simply performs a depth-first preorder walk from city 0.

Algorithm 5.4 DOUBLE-TREE(D)

```
T ← PRIM-MST(D)
W ← the order in which a depth-first (preorder) traversal of T, started at city 0,
first visits each city
return W          (equivalent to an Euler tour of the doubled tree with repeated
cities skipped)
```

Theorem 5.1 (2-approximation). If the cost matrix is symmetric and satisfies the triangle inequality, the double-tree tour H satisfies $C(H) \le 2 \cdot \mathrm{OPT}$ [28].

Proof. The Eulerian circuit of the doubled tree has weight exactly $2 \cdot w(T)$. By the triangle inequality, shortcutting does not increase the weight, so $C(H) \le 2 \cdot w(T)$. By Lemma 5.1, $w(T) \le \mathrm{OPT}$. Therefore $C(H) \le 2 \cdot \mathrm{OPT}$. ∎

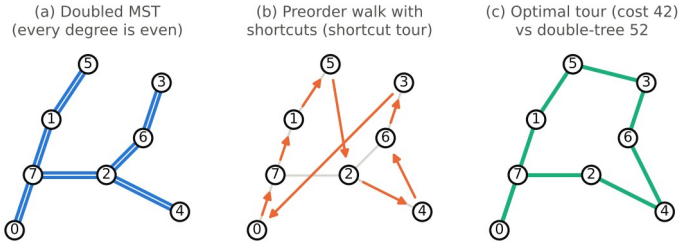


Figure 5.2: Double-tree algorithm on the demonstration instance. (a) The doubled MST (double lines). (b) The preorder walk from city 0 with repeated cities skipped gives a tour of cost 52. (c) The optimal tour, cost 42, for comparison. The observed ratio 52/42 ≈ 1.24 is well below the proven bound of 2.

Table 5.3: Summary of the double-tree algorithm

Item	Detail
Input / output	**Metric symmetric** cost matrix; a Hamiltonian cycle of cost ≤ 2·OPT.
Time / space	$O(n^2)$ time (dominated by Prim's algorithm on a dense matrix); $O(n)$ extra space.
Guarantee	Ratio 2 (Theorem 5.1).
Assumptions	Symmetric costs and triangle inequality. **Both are required**; on arbitrary matrices the output is a valid tour but carries no guarantee.
Strengths	Very fast; easy to prove; deterministic; works with any MST algorithm.
Limitations	Ratio 2 is weak; observed gaps are typically 16–40 % above the optimum or lower bound in Chapter 7, worse than much simpler insertion methods.
Applications	Teaching; quick lower-bound-based estimates; settings where a worst-case guarantee is required but speed is crucial.

5.5 Christofides' Algorithm

Concept. Doubling every tree edge wastes cost. Christofides observed that only the vertices of *odd* degree in the MST prevent it from being Eulerian. Instead of doubling all edges, add a *minimum-weight perfect matching* between the odd-degree vertices; this makes every degree even at a much smaller cost [15], [16].

Steps. (1) Build an MST T. (2) Let O be the set of vertices with odd degree in T; by the handshake lemma, $|O|$ is even. (3) Compute a minimum-weight perfect matching M on O using the costs d_{ij} (Edmonds' blossom algorithm solves this in polynomial time [41]). (4) Form the multigraph $H = T \cup M$; every vertex of H has even degree and H is connected. (5) Find an Eulerian circuit of H. (6) Shortcut repeated vertices to obtain a Hamiltonian cycle.

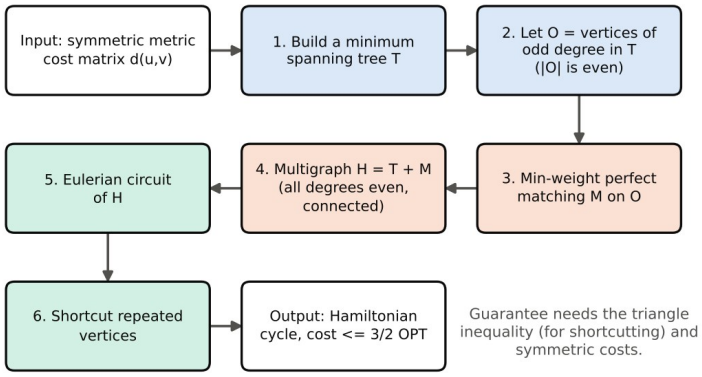


Figure 5.3: Workflow of Christofides' algorithm. The shaded boxes correspond to the six steps; the note on the right states the assumptions required by the guarantee.

Algorithm 5.5 CHRISTOFIDES(D)

```
T ← PRIM-MST(D)
O ← the set of cities having odd degree in T            (|O| is even)
M ← a minimum-weight perfect matching of the cities in O, with edge costs D
H ← the multigraph T ∪ M                                (all degrees even, connected)
C ← an Eulerian circuit of H                            (Hierholzer's algorithm)
tour ← C with every city after its first occurrence removed
return tour
```

Theorem 5.2 (3/2-approximation). If the cost matrix is symmetric and satisfies the triangle inequality, Christofides' tour H' satisfies $C(H') \le (3/2)\cdot\mathrm{OPT}$ [15], [16].

Proof. By Lemma 5.1, $w(T) \le \mathrm{OPT}$. Consider an optimal tour and shortcut it so that it visits only the cities of O, in the same order; by the triangle inequality this closed tour on O has cost at most OPT. Because $|O|$ is even, its edges can be split into two perfect matchings of O by taking alternate edges. The lighter of the two has weight at most OPT/2, and M is a minimum-weight perfect matching, so $w(M) \le \mathrm{OPT}/2$. The Eulerian circuit of H has weight $w(T) + w(M) \le \mathrm{OPT} + \mathrm{OPT}/2 = (3/2)\cdot\mathrm{OPT}$, and shortcutting does not increase it under the triangle inequality. ∎

Worked example. On the demonstration instance (Chapter 7) the MST has weight 31, the odd-degree vertices are {0, 2, 3, 4, 5, 7} and the minimum-weight perfect matching (0,7), (2,4), (3,5) has weight 15. The Eulerian multigraph therefore has weight 46, and after shortcutting the tour costs 42, which here equals the optimum 42. The two lemmas can be checked numerically: 31 ≤ 42 (MST ≤ OPT) and 15 ≤ 21 (matching ≤ OPT/2).

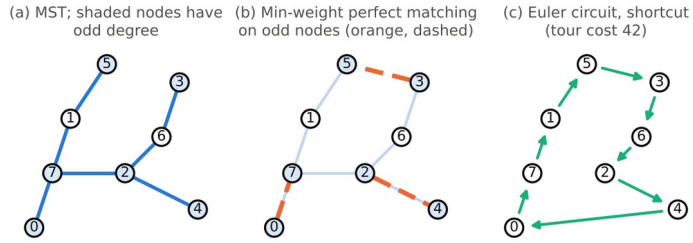


Figure 5.4: Christofides' algorithm on the demonstration instance. (a) The MST; shaded vertices have odd degree. (b) A minimum-weight perfect matching on the odd vertices (orange, dashed). (c) The Eulerian circuit after shortcutting, a tour of cost 42.

Table 5.4: Summary of Christofides' algorithm

Item	Detail
Input / output	**Metric symmetric** cost matrix; a Hamiltonian cycle of cost ≤ (3/2)·OPT.
Time / space	$O(n^3)$, dominated by minimum-weight perfect matching [41]; $O(n^2)$ space for a dense implementation. (The Chapter 7 code uses an exact bitmask matching for at most 14 odd vertices and the NetworkX matching routine otherwise [42].)
Guarantee	Ratio 3/2 (Theorem 5.2). A randomised algorithm with ratio $3/2 - \varepsilon$, $\varepsilon > 10^{-36}$, is now known for metric TSP [11].
Assumptions	Symmetric costs and triangle inequality, as in the double-tree algorithm.
Strengths	Best classical ratio; observed gaps of about 5–11 % in Chapter 7, clearly better than the double-tree.
Limitations	Matching step is expensive and complex to code; quality in practice is inferior to good local search; guarantee lost on non-metric inputs.
Applications	Theory and teaching; settings needing a certified quality bound; starting tours for local search [26].

5.6 Local Search: 2-opt, Or-opt and 3-opt

Concept and motivation. Local search improves an existing tour by small modifications. It is motivated by the observation that tours with crossing edges are rarely optimal. The set of tours reachable by one modification is the **neighbourhood**; a tour with no improving neighbour is a **local optimum**.

2-opt. Remove two edges (a, b) and (c, d) from the tour and reconnect the two resulting paths with (a, c) and (b, d); this reverses the tour section between b and c [27]. The change in cost is $\Delta = d_{ac} + d_{bd} - d_{ab} - d_{cd}$, and the move is applied if $\Delta < 0$. The neighbourhood has $O(n^2)$ tours. Figure 5.5 shows a crossing pair and its repair. **Or-opt** moves a segment of one to three consecutive cities (optionally reversed) to another place in the tour; it is a restricted form of 3-opt. **3-opt** removes three edges and reconnects the three resulting paths in the best of the four ways that use all three new edges; its neighbourhood has $O(n^3)$ tours.

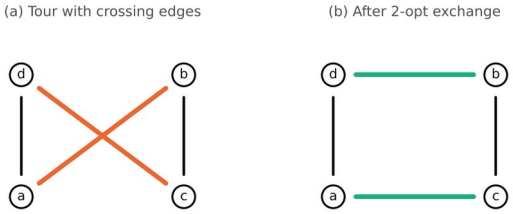


Figure 5.5: The 2-opt move. (a) Edges (a,b) and (c,d) of the tour cross. (b) Replacing them by (a,c) and (b,d) removes the crossing and, for Euclidean distances, shortens the tour.

Algorithm 5.6 TWO-OPT(D, tour)

```
repeat
   improved ← false
   for i ← 0 to n − 2 do
      for j ← i + 2 to n − 1 do                    (skip the pair i = 0, j = n − 1)
         a ← tour[i];  b ← tour[i+1];  c ← tour[j];  d ← tour[(j+1) mod n]
         if D[a][c] + D[b][d] < D[a][b] + D[c][d] then
            reverse the section tour[i+1 .. j];  improved ← true
until improved = false
return tour
```

Table 5.5: Summary of 2-opt and 3-opt

Item	Detail
Input / output	A **symmetric** cost matrix and a starting tour; a 2-opt (or 3-opt) local optimum no more expensive than the start.
Time / space	One pass of 2-opt is $O(n^2)$; of 3-opt $O(n^3)$. The number of passes is not bounded by a small polynomial in the worst case; in practice it is small [26]. $O(n)$ space.
Guarantee	**None** (no constant factor). Worst-case analyses show that local optima of k-opt can be far from optimal [29]. A good result on a test set says nothing about the worst case.
Assumptions	Symmetric costs (the reversal of a section must have the same cost); no metric assumption for the algorithm to run.
Strengths	Simple; fast; improves any starting tour; Chapter 7 shows 2-opt + Or-opt reaching about 2–6 % above the lower bound for $n \geq 30$.
Limitations	Stops at local optima; result depends on the starting tour and move order; 3-opt is much slower.
Applications	Post-processing of constructive tours; component of Lin–Kernighan and iterated local search.

5.7 Lin–Kernighan-Style Methods

Concept. 2-opt and 3-opt use a fixed number of exchanged edges. Lin and Kernighan [17] observed that the best number of edges to exchange is not known in advance and proposed a *variable-depth* search. The search removes an edge, adds a new edge from one of its endpoints, removes a further edge to keep a Hamiltonian path, and so on, building a chain of exchanges. After each step the algorithm computes the cumulative **gain** (cost removed minus cost added) and continues only while the partial gain stays positive; it finally applies the prefix of the chain that gave the best total gain. Edges that have been added may not be removed again, which guarantees termination of one search.

Helsgaun's implementation, LKH, replaces the basic move by a sequential 5-opt move and restricts the choice of new edges to a small candidate set based on *α-nearness*, a measure derived from minimum 1-trees [18], [21]. Johnson and McGeoch's experiments show that Lin–Kernighan variants give clearly better tours than 2-opt and 3-opt at moderate additional cost [26].

Table 5.6: Summary of Lin–Kernighan-style methods

Item	Detail
Input / output	Symmetric cost matrix (usually a metric such as Euclidean distance) and a starting tour; a locally optimal tour with respect to the variable-depth neighbourhood.
Time / space	Depends strongly on implementation (neighbour lists, tour data structures); no simple closed form is claimed here. See [26], [18].
Guarantee	**None** in the worst case. Its excellent performance is empirical.
Strengths	Among the best practical heuristics; scales to very large instances with good data structures.
Limitations	Complex to implement correctly; our project does *not* implement it, so Chapter 7 contains no measurement for it. Results reported in the literature must not be mixed with our measurements.

Empirical effectiveness versus guarantee. The fact that Lin–Kernighan-style methods produce tours within a few percent of optimal on benchmark instances [26], [18] is an *experimental* observation. It is not a proven approximation ratio, and Section 5.6 explains why local-search methods have none.

5.8 Metaheuristics

Metaheuristics are general search frameworks. They are not specific to the TSP, which is why they need *representation* and *neighbourhood* choices. For a tour, the natural representation is the permutation of cities. We describe the versions implemented in Chapter 7 and their parameters.

5.8.1 Simulated annealing

Simulated annealing (SA) imitates the cooling of a metal [30]. At each step a random 2-opt move is proposed. If it shortens the tour it is accepted; if it lengthens the tour by $\Delta > 0$ it is accepted with probability $e^{-\Delta/T}$, where T is the "temperature". T is lowered gradually, so early on the search roams widely and later it settles into a good region. **Parameters in our code:** start from the nearest-neighbour tour; initial $T = 0.3 \cdot (\text{cost})/n$; geometric cooling so that T falls by a factor 1000 over $2000n + 20{,}000$ proposed moves; fixed random seed.

Algorithm 5.7 SIMULATED-ANNEALING(D, T0, α, iterations)

```
tour ← NEAREST-NEIGHBOUR(D, 0);  best ← tour;  T ← T0
repeat iterations times
   choose positions i < j at random;  Δ ← change in cost of reversing tour[i..j]
   if Δ < 0 or random() < exp(−Δ / T) then apply the reversal
   if cost(tour) < cost(best) then best ← tour
   T ← α · T
return best
```

5.8.2 Genetic algorithm

A genetic algorithm (GA) maintains a *population* of tours, selects parents of good fitness (short tours), combines them and occasionally mutates the offspring [31], [32]. **Parameters in our code:** population 80; 200 generations; tournament selection of size 3; order crossover (OX) applied with probability 0.9; inversion mutation with probability 0.2; 2 elite tours copied unchanged; random initial population; fixed seed. The population has no local-search component, which is why the plain GA scales poorly (Section 7.7).

Algorithm 5.8 GENETIC-ALGORITHM(D, N, G)

```
pop ← N random permutations;  evaluate their costs
repeat G generations
   new ← the best 2 tours of pop (elitism)
   while |new| < N do
      p1, p2 ← tournament selections from pop;  child ← OX(p1, p2) with probability
      0.9, else a copy of p1
      with probability 0.2 reverse a random section of child (inversion mutation)
      add child to new
   pop ← new
return the best tour in pop
```

5.8.3 Ant colony optimization

In the Ant System, artificial ants build tours city by city. An ant at city i chooses the next city j among the unvisited ones with probability proportional to $\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}$, where τ_{ij} is the pheromone on edge (i, j) and $\eta_{ij} = 1/d_{ij}$ is the visibility [33]. After all ants have finished, pheromone evaporates ($\tau \leftarrow (1-\rho)\tau$) and each ant deposits 1/(its tour length) on the edges it used, so shorter tours reinforce their edges more. **Parameters in our code:** $\min(n, 20)$ ants; 50 iterations for $n \le 30$, otherwise 30; $\alpha = 1$, $\beta = 3$, $\rho = 0.5$; fixed seed. The more elaborate Ant Colony System adds a local update and a greedier choice rule [34] and was not implemented.

Algorithm 5.9 ANT-SYSTEM(D, m, iterations, α, β, ρ)

```
τ[i][j] ← m / cost(NEAREST-NEIGHBOUR(D))  for all i ≠ j
repeat iterations times
   for each of the m ants: build a tour, moving from i to an unvisited j with
   probability ∝ τ[i][j]^α · (1/D[i][j])^β
   τ ← (1 − ρ) · τ                                  (evaporation)
   for each ant tour t and each edge (u, v) in t: τ[u][v] += 1/cost(t);  τ[v][u] +=
   1/cost(t)
return the best tour found
```

Table 5.7: Summary of the metaheuristics

Item	Detail
Guarantee	**None** for optimality or approximation ratio. Longer runs or better parameters generally help but nothing is proved [30], [32], [33].

Item	Detail
Time (our code)	SA: O(iterations) moves, each O(n) for the reversal; GA: O($N \cdot G \cdot n$); ant system: O(iterations$\cdot m \cdot n^2$) for tour construction with the dense probability vectors of our code.
Assumptions	SA as implemented uses 2-opt moves and therefore requires symmetric costs; GA and ant system run on asymmetric matrices too, but without any theoretical backing.
Strengths	General; easy to extend with side constraints; randomisation avoids the deterministic traps of local search.
Limitations	Parameter sensitivity; random seeds change results; plain GA and ant system do not scale well without local search (Chapter 7).

5.9 Learning-Based Methods

Concept. Instead of designing a construction rule by hand, a neural network *learns* one from examples. Pointer networks read the cities as a sequence and output a permutation by repeatedly "pointing" to the next city [36]. Bello et al. trained such a network with reinforcement learning using the tour length as the reward, so no optimal tours are needed as training labels [37]. Kool et al. used an attention-based encoder–decoder trained with a REINFORCE baseline for the TSP and several routing variants [38]. Bengio et al. classify the ways of combining learning with classical solvers [39].

- **Potential advantages:** very fast tour generation once the model is trained; the ability to specialise to a recurring instance distribution (for example, daily deliveries in one city); easy GPU parallelism.
- **Limitations:** no worst-case approximation guarantee; training needs many instances and substantial computation; performance may degrade when test instances are larger than or distributed differently from the training instances; results reported in the literature are for specific benchmarks and cannot be transferred to other settings without evidence [39].
- **Computational considerations:** inference cost grows with n (attention over all cities is quadratic); training cost is paid once but can be large.
- **Position in this report:** because of the points above, learning-based methods are classified as heuristics. They were not implemented, and no experimental claim about them is made.

5.10 Method Selection

Table 5.8 relates the choice of method to instance size, the need for guarantees and the available resources. The size ranges are indicative, because the true limits depend on the implementation language, hardware and instance structure; for orientation, our pure-Python dynamic program needed about 0.5 s at 16 cities (Table 7.2).

Table 5.8: Choosing a method

Situation	Suitable methods	Reason
Very small instances (up to about 12–16 cities in simple code)	Held–Karp dynamic programming; brute force as a cross-check	Optimal answer is affordable; also needed to validate heuristics.
Small to medium instances where optimality matters	Branch and bound / branch and cut using an established solver [3]	Provable optimality for sizes far beyond dynamic programming, no polynomial-time promise.
A certified worst-case quality is required (metric instance)	Christofides (3/2); double-tree (2) if speed is crucial	Proven ratio; check the triangle inequality first.
Good tours quickly, moderate size (hundreds to a few thousand cities)	Farthest/cheapest insertion followed by 2-opt + Or-opt; or Lin–Kernighan-style search	Best observed quality-per-time in Chapter 7 and in [26].
Maximum quality in available time, large instances	Lin–Kernighan variants such as LKH [18]; metaheuristics seeded with good tours	Strongest practical results reported in the literature.
Asymmetric or non-metric costs	Exact methods for small n; nearest neighbour, cheapest insertion, GA or ant system for larger n; ATSP theory [12]	MST-based and 2-opt methods need symmetry; metric guarantees do not apply.
Euclidean instance and theoretical accuracy target	PTAS of Arora [20] (theory)	(1+ε) guarantee in fixed dimension; of theoretical interest.

Reminder. Never apply a guarantee proved for metric symmetric TSP to an arbitrary cost matrix, and never present an observed gap on a test set as an approximation ratio.

Chapter 6: Comparative Analysis

This chapter compares the methods of Chapter 5 on the criteria requested by the project: theoretical guarantee, assumptions, strengths, limitations, runtime and memory, implementation effort and suitability for different instance sizes. The comparison is drawn from the literature and from the analysis of Chapter 5; the project's own measurements are collected in Chapter 7 and referred to where they illustrate a point.

6.1 Guarantees, Assumptions, Strengths and Limitations

Table 6.1: Comparison of methods: category, assumptions, guarantee, strengths and limitations

Method	Category	Required assumptions	Optimality / approximation guarantee	Typical strengths	Main limitations
Brute force	Exact	None	Optimal	Trivially correct; reference answer	$(n-1)!/2$ tours; hopeless beyond about 11–12 cities
Held–Karp DP	Exact	None (works for asymmetric costs)	Optimal	$O(n^2 2^n)$, far faster than brute force	$O(n2^n)$ memory
Branch and bound / cut	Exact	Bound-dependent (1-tree needs symmetry)	Optimal	Handles far larger n with good bounds [3]	Exponential worst case; complex solver
Nearest neighbour	Constructive heuristic	None to run	Metric: $O(\log n)$ ratio only [28]	Simplest and fastest	Poor tours; no constant ratio
Cheapest insertion	Constructive heuristic	Metric symmetric for bound	Ratio 2 (metric) [28]	Better than NN; adaptable	Slower than NN; greedy
Farthest insertion	Constructive heuristic	None to run	None claimed here	Best constructive tours in our tests	No proven constant; slower than NN
Double-tree	Approximation	Metric symmetric	Ratio 2	Fast; simple proof	Weak ratio; mediocre tours
Christofides	Approximation	Metric symmetric	Ratio 3/2	Best classical ratio	Matching is costly; guarantee lost off-metric
2-opt / Or-opt / 3-opt	Local search	Symmetric costs	None	Large improvement for little effort	Local optima; no worst-case bound [29]
Lin–Kernighan / LKH	Local search	Symmetric costs	None	Best practical quality [26], [18]	Complex; not implemented here
Simulated annealing	Metaheuristic	Symmetric (2-opt moves)	None	General; escapes local minima	Parameter tuning
Genetic algorithm	Metaheuristic	None	None	Flexible	Weak scaling when plain
Ant colony	Metaheuristic	None	None	Adaptive; parallel	Slow; parameter sensitive
Learned models	Learning-assisted	Training distribution	None	Fast inference [38]	Generalisation; training cost

6.2 Time, Memory and Practical Considerations

Table 6.2: Time, memory, implementation effort and suitable sizes

Method	Time	Extra memory	Implementation effort	Suitable size (indicative)
Brute force	$O(n!)$	$O(n)$	Very low	Up to about 10–11 cities in simple code
Held–Karp DP	$O(n^2 2^n)$	$O(n2^n)$	Low	Up to about 15–20 cities
Nearest neighbour	$O(n^2)$	$O(n)$	Very low	Any size
Insertion heuristics	$O(n^3)$ simple; $O(n^2 \log n)$ tuned	$O(n)$	Low	Up to thousands
Double-tree	$O(n^2)$	$O(n)$	Low	Up to many thousands
Christofides	$O(n^3)$	$O(n^2)$	High (matching)	Up to thousands
2-opt / Or-opt	$O(n^2)$ per pass	$O(n)$	Low to medium	Up to many thousands (neighbour lists help)
3-opt	$O(n^3)$ per pass	$O(n)$	Medium	Up to a few hundred in simple code
Lin–Kernighan	Implementation dependent	$O(n)$ to $O(n^2)$	High	Very large instances [18]
SA / GA / ACO	Budget-controlled	$O(n)$ to $O(n^2)$	Medium	Small to medium without local search

"Indicative" sizes depend on the implementation and the machine and are not measurements of this project except where Chapter 7 says so. Memory is counted in addition to the $n \times n$ cost matrix itself, which already needs $O(n^2)$ space.

6.3 Guarantee Versus Observed Performance

A formal approximation ratio is a **supremum over all instances of a class**; an observed gap is a **number measured on particular instances**. They answer different questions. Figure 6.1 plots, for 300 random *metric* instances of 9 cities, the largest observed tour-to-optimum ratio of three methods next to the proven bounds. The observed ratios sit far below the guarantees (Christofides 1.23 against 3/2, double-tree 1.46 against 2), which is what one expects because worst-case instances are specially constructed and rare. However, the figure does **not** prove anything about the guarantee: 300 random instances cannot establish a worst-case statement, and the nearest-neighbour bar shows an instance-sample statistic with no proven constant bound at all.

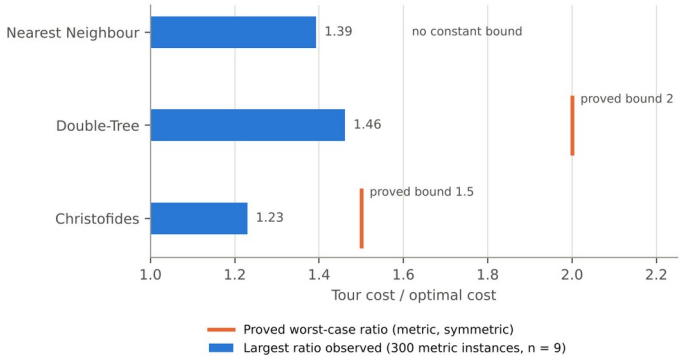


Figure 6.1: Largest tour-to-optimum ratio observed over 300 random metric instances (n = 9, Chapter 7) compared with the proved worst-case ratios for metric symmetric TSP. The bars are measurements; the orange marks are theorems.

6.4 No Method Is Universally Best

- **If optimality is mandatory,** only exact methods qualify, and the size limit is set by the exponential growth of their cost.
- **If a worst-case certificate is mandatory,** only the approximation algorithms qualify, and they require a metric symmetric instance; their typical tours are worse than those of simple local search.
- **If the best practical tour in modest time is the aim,** local search (and Lin–Kernighan variants) is preferred, but it comes with no certificate.
- **If the problem has extra constraints** (time windows, capacities), metaheuristics and insertion-based heuristics are easier to adapt than the MST-based algorithms.
- **If very large instances must be solved quickly,** simple constructive or space-filling methods may be the only affordable choice.

The trade-off is therefore between *certainty* (optimality or a proven bound), *quality* (observed tour length), *speed*, *memory* and *implementation effort*, and each method occupies a different position. The experiments of Chapter 7 illustrate this: Christofides has a proven 3/2 bound but, on our instances, is beaten by farthest insertion and by 2-opt + Or-opt, which have no such bound.

Chapter 7: Demonstration and Experimental Study

Experimental integrity. Every number in this chapter was produced by running the programme `tsp_survey.py` (supplied with the report) and was inserted into this document from its output file `results.json` by script. Nothing was typed by hand or estimated. Run times are wall-clock measurements on the machine described in Table 7.1 and will differ on other machines; costs and tours are repeatable because all random numbers use fixed seeds.

7.1 Software Design

The programme is a single Python file organised into six groups of functions (Figure 7.1). *Instance* functions create or validate cost matrices; *exact*, *constructive*, *approximation*, *local-search* and *metaheuristic* groups implement the methods of Chapter 5; and a *harness* runs a method, validates its output and records cost and time. The principal data structures are: a NumPy array for the cost matrix; a Python list of city indices for a tour; a list of edge pairs for a spanning tree; adjacency lists for the Eulerian-circuit step; and bitmask-indexed tables for the Held–Karp and matching dynamic programs.

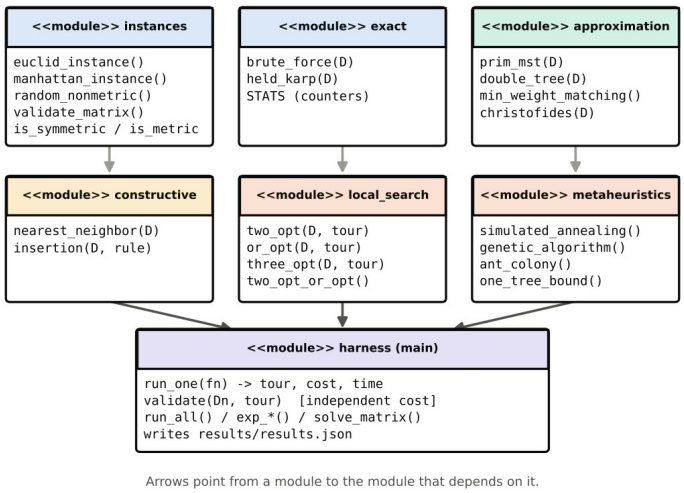


Figure 7.1: Module structure of tsp_survey.py (UML-style class/module diagram). Each box lists the main functions of one group; arrows point to the module that depends on it.

Figure 7.2 shows the experimental workflow. A matrix is loaded or generated, validated, passed to the applicable algorithms, and every returned tour is checked by `validate()`, which confirms that the tour is a permutation of all cities (so every city is visited exactly once and the cycle closes at the start) and **recomputes its cost independently** from the matrix with NumPy. A tour that fails validation raises an error, so no invalid result can enter the tables.

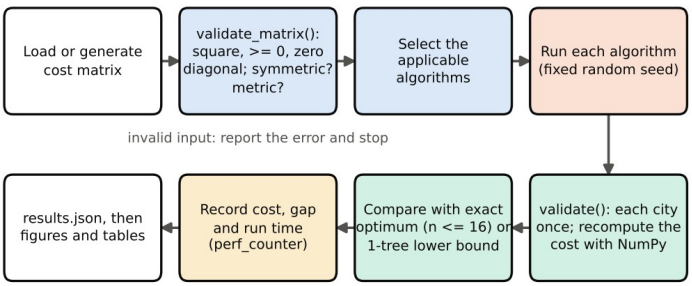


Figure 7.2: Flowchart of the experimental procedure, including input validation and independent re-computation of every tour cost.

7.2 Input Format, Validation and How to Run

Supported input. A plain-text CSV file containing an $n \times n$ cost matrix, one row per line, comma-separated. **Validation** rejects a matrix that is not square, has fewer than 4 cities, contains non-numeric, NaN or infinite entries, has negative entries or a non-zero diagonal. The programme then *detects* whether the matrix is symmetric and metric and chooses the applicable algorithms: for an asymmetric matrix it skips MST-based methods, 2-opt, Or-opt,

3-opt and simulated annealing; for a symmetric non-metric matrix it prints a warning that the 2 and 3/2 guarantees do not apply.

Execution. After installing Python 3 with `numpy` and `networkx`:

Commands

```
python3 tsp_survey.py --matrix sample/demo_8city.csv     # solve one matrix with all applicable methods
python3 tsp_survey.py --out results                      # full experiments, writes results/results.json
python3 tsp_survey.py --out results_quick --quick        # fast smoke test
python3 make_figures.py                                  # regenerate all figures from results.json
```

Sample output. The first lines printed for the 8-city demonstration matrix are shown below (a complete listing is in Appendix D). Invalid input produces a message such as `INPUT ERROR: diagonal must be zero (cost of staying in a city)` and exit code 2.

Output of: python3 tsp_survey.py --matrix sample/demo_8city.csv (excerpt; times from one run)

```
Loaded 8x8 matrix: symmetric=True, metric=True
Method                          Cost    Gap%   Time(ms)  Tour
Held-Karp DP (exact)           42.00    0.00      0.368  [0, 7, 2, 4, 6, 3, 5, 1, 0]
Brute Force (exact)            42.00    0.00      1.226  [0, 1, 5, 3, 6, 4, 2, 7, 0]
Nearest Neighbour              42.00    0.00      0.008  [0, 7, 1, 5, 3, 6, 2, 4, 0]
Cheapest Insertion             42.00    0.00      0.020  [0, 4, 2, 6, 3, 5, 1, 7, 0]
Farthest Insertion             42.00    0.00      0.024  [0, 7, 1, 5, 3, 6, 4, 2, 0]
Double-Tree                    52.00   23.81      0.044  [0, 7, 1, 5, 2, 4, 6, 3, 0]
```

7.3 Experimental Environment and Settings

Table 7.1: Experimental environment and settings

Item	Value
Language and libraries	Python 3.13.18, NumPy 2.5.3 [43], NetworkX [42] (used only for matching when more than 14 odd-degree vertices occur)
Machine	Linux-6.18.44-fc-v233-x86_64-with-glibc2.39; 2 CPU cores; single-threaded pure-Python algorithms
Timing	`time.perf_counter()`; median of 3 runs for the fast deterministic methods, single run for the randomised and slow methods
Instances	Uniform random points in a 100×100 square with Euclidean distance (metric), fixed seeds; the 8-city Manhattan demonstration instance; random symmetric non-metric matrices (integer weights 1–100) for the guarantee test
Reference values	Exact optimum (Held–Karp) for $n \le 15$; Held–Karp 1-tree lower bound [21] for larger n
Metaheuristic budgets	SA: $2000n + 20{,}000$ moves. GA: population 80, 200 generations. Ant system: $\min(n, 20)$ ants, 50 ($n \le 30$) or 30 iterations. Same settings for every instance; **no tuning**
Self-tests (all passed)	brute force == Held-Karp on 30 instances (n=8); bitmask matching == networkx matching on 20 instances (14 vertices); all algorithms return valid tours for n = 4..7; no heuristic cost is below the exact optimum (10 instances, n=9); Held-Karp 1-tree bound <= optimum (10 instances, n=10)
Total run time of all experiments	55.1 s

7.4 Exact Methods: Brute Force Versus Dynamic Programming

Table 7.2 compares the two exact methods. "Candidates" counts the complete tours examined by brute force and the DP transitions examined by Held–Karp, so the table shows both the *count* and the *time*. Brute force examines $(n-1)!/2$ tours; Held–Karp examines $n^2 2^n$-type numbers of transitions.

Table 7.2: Exact methods: candidates examined and mean run time (ms)

n	Brute-force tours	Brute-force time (ms)	DP transitions	DP time (ms)
4	3	0.005	12	0.012
5	12	0.007	48	0.051
6	60	0.035	160	0.086
7	360	0.172	480	0.130
8	2,520	1.33	1,344	0.446
9	20,160	10.79	3,584	0.875
10	181,440	102	9,216	2.08
11	1,814,400	1148	23,040	5.48
12	–	–	56,320	12.51

n	Brute-force tours	Brute-force time (ms)	DP transitions	DP time (ms)
13	–	–	135,168	31.71
14	–	–	319,488	77.62
15	–	–	745,472	191
16	–	–	1,720,320	451

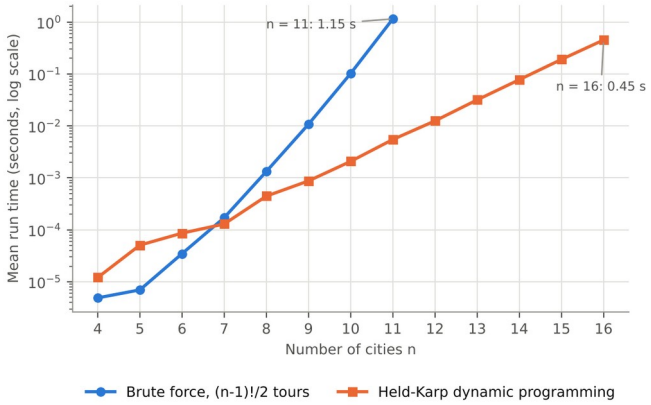


Figure 7.3: Measured run time of the two exact methods against the number of cities (log scale). Brute force grows factorially and was run only up to 11 cities; dynamic programming was run up to 16.

At 11 cities brute force needs 1148 ms against 5.48 ms for the dynamic program, a ratio of about 209. Each additional city multiplies the brute-force effort by about n but the DP effort by only about 2. Even so, the dynamic program takes 451 ms at 16 cities in this pure-Python implementation and its memory grows as $O(n2^n)$, so it too is limited to small instances. These measurements support the motivation for approximation methods but say nothing about the quality of approximation methods.

7.5 Worked Demonstration Instance

The demonstration instance has 8 cities at the integer points listed in Table 7.3, with Manhattan distances (a metric). The optimal tour is known from brute force and the dynamic program, which agree.

Table 7.3: Demonstration instance: city coordinates and cost matrix (Manhattan distance)

City	0	1	2	3	4	5	6	7
0 (0,0)	0	8	8	16	10	13	12	4
1 (2,6)	8	0	6	8	12	5	6	4
2 (5,3)	8	6	0	8	6	7	4	4
3 (8,8)	16	8	8	0	8	5	4	12
4 (9,1)	10	12	6	8	0	13	6	10
5 (4,9)	13	5	7	5	13	0	7	9
6 (7,5)	12	6	4	4	6	7	0	8
7 (1,3)	4	4	4	12	10	9	8	0

Table 7.4: Results on the 8-city instance (optimal cost 42). All tours validated; costs recomputed independently

Method	Cost	Gap (%)	Time (ms)	Tour
Brute Force (exact)	42	0.00	1.23	0–1–5–3–6–4–2–7–0
Held-Karp DP (exact)	42	0.00	0.356	0–7–2–4–6–3–5–1–0
Nearest Neighbour	42	0.00	0.004	0–7–1–5–3–6–2–4–0
Cheapest Insertion	42	0.00	0.013	0–4–2–6–3–5–1–7–0
Farthest Insertion	42	0.00	0.016	0–7–1–5–3–6–4–2–0
Double-Tree	52	23.81	0.014	0–7–1–5–2–4–6–3–0
Christofides	42	0.00	0.036	0–7–1–5–3–6–2–4–0
2-opt (from NN)	42	0.00	0.007	0–7–1–5–3–6–2–4–0
Or-opt (from NN)	42	0.00	0.033	0–7–1–5–3–6–2–4–0
2-opt+Or-opt (from NN)	42	0.00	0.067	0–7–1–5–3–6–2–4–0
3-opt (from NN)	42	0.00	0.035	0–7–1–5–3–6–2–4–0
Simulated Annealing	42	0.00	51.02	0–7–1–5–3–6–2–4–0

Method	Cost	Gap (%)	Time (ms)	Tour
Genetic Algorithm	42	0.00	111	1–7–0–2–4–6–3–5–1
Ant Colony (AS)	42	0.00	17.02	5–3–6–2–4–0–7–1–5

Of the 14 rows, 13 reach the optimum of 42. The only exception is the double-tree algorithm with cost 52 (gap 23.8 %), which is still inside its guaranteed ratio of 2. Different tours with the same cost (for example brute force versus Held–Karp) are different starting points or directions of the same optimal cycle, or alternative optimal cycles. One small instance cannot rank the methods; Sections 7.6 and 7.7 use many instances.

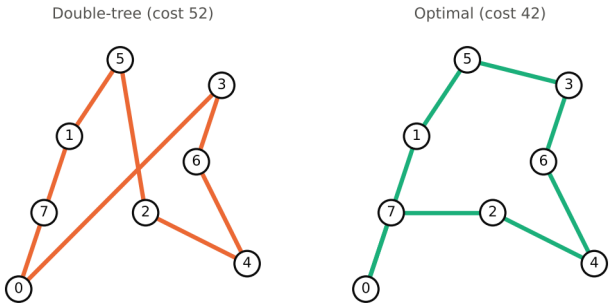


Figure 7.4: The double-tree tour (cost 52, left) and the optimal tour (cost 42, right) on the demonstration instance.

7.6 Solution Quality on Small Instances (Optimum Known)

Table 7.5 compares each method with the exact optimum on 30 random instances of 10 cities and 10 random instances of 15 cities. The **gap** of a tour is 100·(cost − OPT)/OPT. "Optimal" counts the instances on which the method found a tour of optimal cost.

Table 7.5: Gap to the exact optimum (%) on random Euclidean instances

Method	n=10: mean	n=10: max	n=10: optimal	n=15: mean	n=15: max	n=15: optimal
Nearest Neighbour	7.75	19.6	5/30	14.33	30.6	0/10
Cheapest Insertion	5.42	25.8	6/30	9.43	22.5	1/10
Farthest Insertion	0.69	8.2	19/30	0.96	7.0	5/10
Double-Tree	17.65	35.2	0/30	23.46	34.5	0/10
Christofides	4.56	16.8	8/30	7.13	14.3	2/10
2-opt (from NN)	0.64	8.6	23/30	1.74	6.6	4/10
Or-opt (from NN)	0.00	0.0	30/30	0.20	2.0	9/10
2-opt+Or-opt (from NN)	0.00	0.0	30/30	0.27	2.0	8/10
3-opt (from NN)	0.00	0.0	30/30	0.20	2.0	9/10
Simulated Annealing	0.00	0.0	30/30	0.00	0.0	10/10
Genetic Algorithm	0.03	0.9	29/30	1.15	4.8	5/10
Ant Colony (AS)	0.05	1.4	29/30	0.00	0.0	10/10

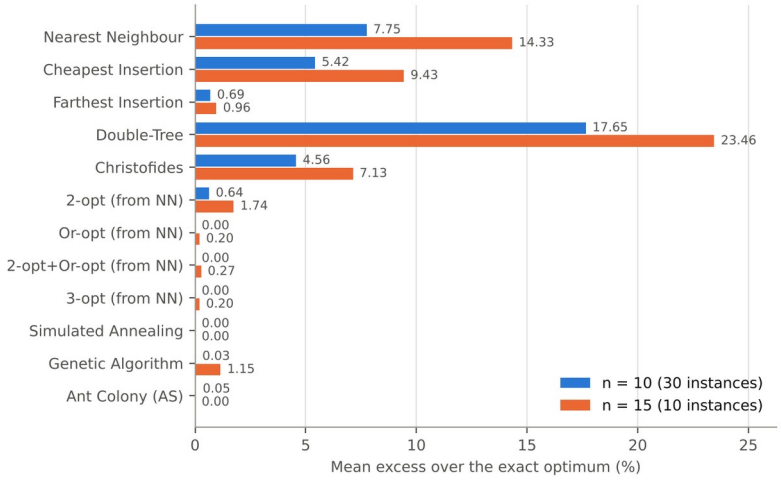


Figure 7.5: Mean gap to the exact optimum for each method on the small random instances of Table 7.5.

The table shows a clear ordering among the *constructive* methods: farthest insertion (mean gap 0.96 % at n = 15) is far better than cheapest insertion (9.43 %), nearest neighbour (14.33 %) and the double-tree (23.46 %). Christofides (7.13 %) is better than the double-tree, as its better ratio suggests, but not better than farthest insertion. The local-search methods reduce the gap sharply; at n = 10 Or-opt, 2-opt + Or-opt, 3-opt and simulated annealing reached the optimum on all 30 instances. Because only 30 and 10 instances were used, differences of a few tenths of a percent between the strong methods are not statistically meaningful.

The 1-tree lower bound averaged 100.00 % of the optimum at n = 10 and 99.83 % at n = 15 (minimum 98.48 %), which justifies using it as a stand-in for the optimum in the next section.

7.7 Scaling Up to 200 Cities

For larger instances the optimum is unknown, so tours are compared with the Held–Karp 1-tree lower bound [21], which is computed by subgradient optimisation seeded with the best tour found. The tables report the mean over 3 random instances per size. "Excess over bound" is 100·(cost − LB)/LB; because LB ≤ OPT, the true gap to the optimum is *smaller* than the figure shown. A dash means that the method was not run (3-opt is O(n^3) per pass and was run only for $n \le 100$).

Table 7.6: Mean run time (ms) against number of cities

Method	n=10	n=20	n=30	n=50	n=75	n=100	n=150	n=200
Nearest Neighbour	0.006	0.020	0.036	0.112	0.241	0.499	1.02	1.77
Cheapest Insertion	0.022	0.157	0.450	2.21	7.64	16.09	51.61	154
Farthest Insertion	0.026	0.135	0.312	1.32	4.04	7.71	32.34	67.33
Double-Tree	0.019	0.067	0.103	0.295	0.617	1.05	2.55	3.41
Christofides	0.040	0.165	1.18	7.56	12.81	37.84	84.85	220
2-opt (from NN)	0.016	0.092	0.176	0.543	1.68	2.70	7.91	15.85
Or-opt (from NN)	0.098	0.467	1.16	3.00	15.28	21.16	65.30	80.05
2-opt+Or-opt (from NN)	0.132	0.826	1.52	4.30	14.95	18.19	62.12	98.99
3-opt (from NN)	0.140	1.67	5.89	37.87	155	352	–	–
Simulated Annealing	55.49	99.01	125	188	326	471	599	681
Genetic Algorithm	129	133	159	193	267	317	380	498
Ant Colony (AS)	30.31	103	167	161	252	358	651	845

Table 7.7: Mean excess over the Held–Karp lower bound (%)

Method	n=10	n=20	n=30	n=50	n=75	n=100	n=150	n=200
Nearest Neighbour	14.5	16.0	21.5	21.5	28.7	32.3	26.9	34.6
Cheapest Insertion	9.0	11.2	10.8	15.8	20.2	17.2	22.1	21.1
Farthest Insertion	4.0	5.8	5.1	4.0	8.1	8.3	9.3	8.8
Double-Tree	16.2	25.0	25.2	39.9	40.2	38.3	36.8	38.4
Christofides	4.6	8.0	9.6	10.2	15.0	10.1	10.4	11.0
2-opt (from NN)	1.4	3.3	3.6	8.5	12.8	4.9	11.1	8.5
Or-opt (from NN)	0.0	0.1	6.5	6.9	10.1	9.5	10.0	11.7
2-opt+Or-opt (from NN)	0.0	0.1	2.1	2.6	5.4	2.8	5.8	5.2
3-opt (from NN)	0.2	0.1	3.1	1.8	3.2	2.9	–	–
Simulated Annealing	0.0	0.1	1.8	3.6	5.0	5.0	6.1	6.2
Genetic Algorithm	0.0	0.1	2.5	24.3	67.2	115.9	221.4	329.2
Ant Colony (AS)	0.0	0.8	2.9	6.5	12.0	13.0	20.6	18.5

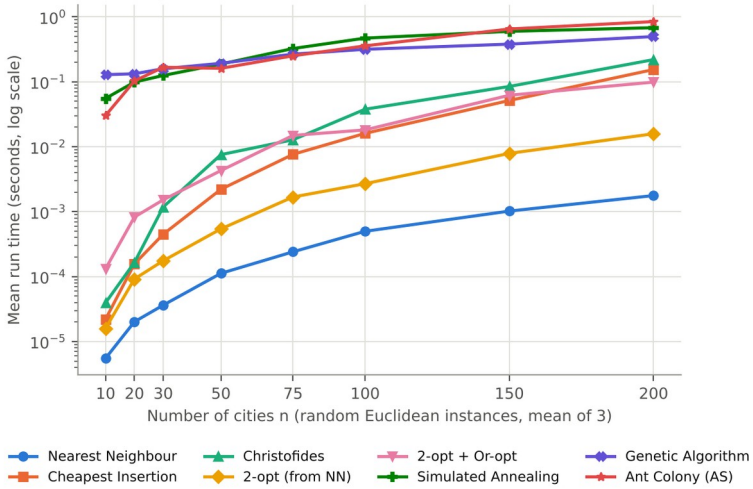


Figure 7.6: Mean run time against number of cities for eight representative methods (logarithmic vertical axis).

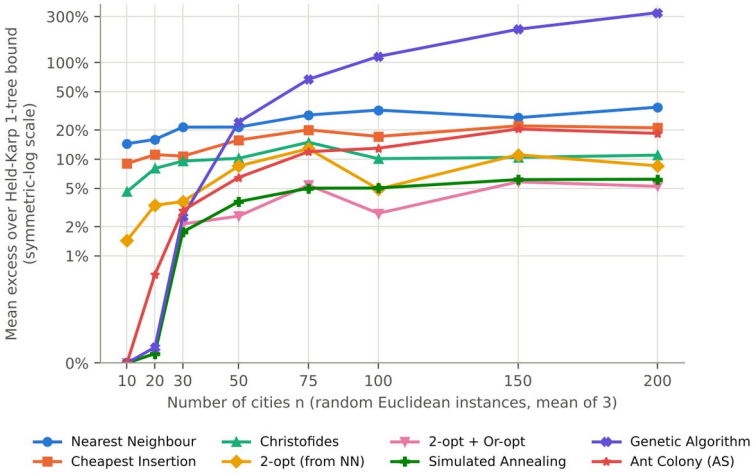


Figure 7.7: Mean excess over the Held–Karp lower bound against number of cities (symmetric-logarithmic vertical axis, because the plain genetic algorithm degrades strongly).

Observations. (1) Among the constructive methods farthest insertion is the best at every size (8.8 % at n = 200), followed by Christofides (11.0 %); cheapest insertion (21.1 %), nearest neighbour (34.6 %) and the double-tree (38.4 %) are clearly worse. (2) Adding local search to nearest-neighbour tours helps a lot: 2-opt + Or-opt reaches 5.2 % at n = 200 in 98.99 ms, better than Christofides (220 ms) and better than the time-hungry metaheuristics. (3) Simulated annealing is competitive (6.2 % at n = 200) but needs about 681 ms, versus 98.99 ms for 2-opt + Or-opt. (4) The plain genetic algorithm and, to a lesser degree, the ant system fall far behind as *n* grows (genetic algorithm 329.2 %, ant system 18.5 % at n = 200). With a fixed budget of 200 generations and no local search, the GA has simply not converged; this is a statement about *our untuned settings*, not about genetic algorithms in general. (5) Times grow polynomially for the constructive and local-search methods and are dominated by the fixed budgets for the metaheuristics.

7.8 Do the Guarantees Hold? A Metric Versus Non-Metric Test

Theorems 5.1 and 5.2 hold only for metric symmetric matrices. To see what happens outside this class, each of 300 random instances of 9 cities was solved exactly and by three methods. The metric set used Euclidean distances (0 of 300 instances violated the triangle inequality, as expected). The non-metric set used independent random integer weights; all 300 of 300 matrices violated the triangle inequality. The columns "% above 1.5 / 2" give the percentage of instances whose ratio to the optimum exceeded 1.5 and 2.

Table 7.8: Ratio to the exact optimum over 300 instances of 9 cities

Method	Proved bound (metric)	Metric: mean	Metric: max	Metric: % > 1.5 / > 2	Non-metric: mean	Non-metric: max	Non-metric: % > 1.5 / > 2
Nearest Neighbour	none	1.10	1.39	0.0 / 0.0	1.23	2.36	8.0 / 1.0
Double-Tree	2	1.16	1.46	0.0 / 0.0	1.50	3.46	42.3 / 6.7
Christofides	1.5	1.05	1.23	0.0 / 0.0	1.25	2.18	10.7 / 0.7

On the metric instances no run exceeded its proved bound (largest observed ratios 1.46 for the double-tree and 1.23 for Christofides). This is consistent with the theorems and acts as a check of the implementation; it is **not** a proof, and it does not show that the bounds are tight. On the non-metric matrices the guarantees fail: the double-tree exceeded ratio 2 on 6.7 % of the instances (largest ratio 3.46) and Christofides exceeded 3/2 on 10.7 % of them (largest ratio 2.18). This demonstrates why metric guarantees must never be applied to arbitrary cost matrices. (The failures are not surprising: for such matrices a shortcut can cost much more than the path it replaces.)

7.9 Interpretation and Threats to Validity

- **Literature versus experiment.** The guarantees quoted throughout come from the cited literature; Chapter 7 only checks them on small samples and measures practical behaviour. Observed gaps are not approximation ratios.
- **Sample size.** The scaling experiment uses 3 random instances per size; the quality experiment 30 (n = 10) and 10 (n = 15). Differences between closely ranked methods are not statistically established.
- **Instance type.** Random uniform Euclidean instances are only one family. Clustered or library instances such as TSPLIB [10] may rank methods differently [26].
- **Implementation effects.** Run times come from pure Python on a shared two-core machine and are useful for growth trends, not for comparison with optimised C code or with published solver timings.
- **Untuned metaheuristics.** Parameters were fixed in advance and not optimised; better settings, local-search hybrids or Ant Colony System could change the ranking of GA and ant system.
- **Lower bound.** Excess over the Held–Karp bound overstates the true gap by the (small) gap between bound and optimum, which was at most about 1.5 % in our small-instance tests (Section 7.6).
- **Not implemented.** Lin–Kernighan/LKH, branch and bound and learning-based methods were *not* implemented or measured; statements about them come from the literature only.

Chapter 8: Applications, Limitations and Future Directions

8.1 Applications

Logistics and delivery planning. A courier visiting a set of addresses and returning to a depot faces a TSP; with several vehicles, capacities or time windows it becomes a vehicle-routing problem (VRP), a direct generalisation of the TSP whose study began with Dantzig and Ramser's truck-dispatching problem [44], [14]. Constructive and insertion heuristics are the building blocks of many practical routing systems [14].

Transportation and route planning. Tour planning for sightseeing, school buses, waste collection or inspection routes can be modelled with a cost matrix of travel times. The *shortest-path closure* of a road network is symmetric only if every road is two-way; one-way streets and time-dependent traffic make the matrix asymmetric, which moves the problem from the STSP to the ATSP [12].

Manufacturing and production sequencing. When a machine must process jobs in some order and the set-up time between two jobs depends on the pair, the cost of an order is the length of a tour through the jobs; the TSP and its variants have been used to model such sequencing and scheduling problems [2].

Circuit and wiring optimization. Drilling holes in a printed circuit board, or visiting the points of a chip to test them, requires a short path through a set of points in the plane; the Euclidean or Manhattan metric makes these instances metric, so the algorithms of Chapter 5 apply directly [2], [26].

Robotics and inspection paths. A robot that must visit a set of waypoints (for example inspection or measurement points) has a cost matrix of travel times between points. Obstacles can make distances non-Euclidean, but shortest-path distances through the free space remain symmetric and metric.

Related combinatorial optimization problems. The ideas of Chapter 5 recur in the vehicle-routing problem [14], in Steiner-tree and matching problems (the MST and matching steps of Christofides' algorithm), and in other sequencing problems [2], [9].

8.2 Limitations of the Simplified TSP Model

- **Asymmetric and time-dependent costs.** Travel time depends on direction and time of day; metric symmetric guarantees then do not apply (Chapter 7, Section 7.8).
- **Side constraints.** Capacities, delivery time windows, driver shifts, precedence between stops and multiple depots need richer models such as VRP variants [14]; the MST and matching constructions do not extend simply to them.
- **Uncertainty.** Real travel times and demands vary; a single optimal tour for fixed costs may be fragile.
- **Multiple objectives.** Distance, time, fuel, driver workload and service quality may have to be balanced rather than a single cost minimised.
- **Cost-matrix preparation.** For n stops an $n \times n$ matrix is needed; computing it from a road network can dominate the total effort.

8.3 Future Directions

- **Stronger approximation methods.** Karlin, Klein and Oveis Gharan broke the 3/2 barrier for metric TSP [11] and Svensson, Tarnawski and Végh obtained a constant factor for ATSP [12]; closing the gap to the known inapproximability bounds [23] is an active research area covered in [13].
- **Improved local search.** Candidate-set selection and deeper moves, as in LKH [18], and better data structures for very large instances [26].
- **Hybrid algorithms.** Combining constructive tours, local search and population or annealing frameworks, for example seeding a metaheuristic with Christofides or insertion tours (our experiment shows the plain GA is weak without local search).
- **Parallel computation.** Running many independent searches or ants simultaneously, exploiting multi-core and GPU hardware.
- **Learning-assisted optimization.** End-to-end models [37], [38] and learning-enhanced solvers [39]; the main open questions are generalisation to larger instances and any form of performance guarantee.
- **Larger and more varied benchmarks.** Standard libraries [10] and exact solvers [3] provide reference values against which new methods can be compared fairly.
- **Extensions to vehicle routing and related problems.** Adapting the best TSP methods to constrained routing [14].

Chapter 9: Conclusion

Objective and scope. This report surveyed approximation methodologies for the Travelling Salesman Problem: it formulated the problem as a minimum-cost Hamiltonian cycle search, classified solution approaches by the strength of their promise, analysed the important methods in detail, and illustrated their behaviour with a validated Python implementation.

Key theoretical points. (1) The TSP is NP-hard, and for general cost matrices no constant-factor approximation exists unless P = NP [19], [1]. (2) With the triangle inequality, the double-tree algorithm achieves ratio 2 and Christofides' algorithm ratio 3/2; the proofs use the facts that the MST is a lower bound and that shortcutting is free in a metric [28], [15], [16]. (3) A randomised ratio of $3/2 - \varepsilon$ is now known for metric TSP [11], a constant factor for ATSP [12], and a PTAS for Euclidean instances in fixed dimension [20]. (4) Nearest neighbour, local search, Lin–Kernighan-style methods, metaheuristics and learning-based methods have no constant-factor guarantee for the TSP.

Verified experimental findings. In our experiments (Chapter 7; random Euclidean instances, pure Python, fixed seeds) the dynamic program was faster than brute force by a large factor and both grew exponentially or faster; on all 300 metric test instances the double-tree and Christofides tours respected their proved bounds, while on random non-metric matrices both bounds were violated; among the constructive methods, farthest insertion gave the best tours, with Christofides second; adding 2-opt and Or-opt to nearest-neighbour tours lowered the mean excess over the lower bound at n = 200 from 34.6 % to 5.2 %; the plain genetic algorithm, with our fixed untuned budget, did not scale. These are observations about particular instances and settings and are not approximation ratios.

Practical trade-offs. The methods differ in certainty (optimality, a proven ratio or none), tour quality, speed, memory and implementation effort. A method with the best theoretical guarantee (Christofides) was not the method with the best observed quality, and no method is universally best (Section 6.4).

Limitations of the survey. The literature selection is representative, not exhaustive; Lin–Kernighan/LKH, branch and bound and learning-based methods were not implemented; instance sizes were modest; only random uniform Euclidean instances were used; and the experiments were run on a single machine.

Future work. Implementing Or-opt-based iterated local search and a simple Lin–Kernighan variant, evaluating on TSPLIB instances [10], tuning the metaheuristics, and extending the study to vehicle routing are natural next steps (Section 8.3).

References

[1] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA, USA: W. H. Freeman, 1979.

[2] E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, and D. B. Shmoys, Eds., *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Chichester, U.K.: Wiley, 1985.

[3] D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton, NJ, USA: Princeton Univ. Press, 2006.

[4] G. B. Dantzig, D. R. Fulkerson, and S. M. Johnson, "Solution of a large-scale traveling-salesman problem," RAND Corp., Santa Monica, CA, USA, Paper P-510, Apr. 1954.

[5] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *J. Soc. Ind. Appl. Math.*, vol. 10, no. 1, pp. 196–210, 1962.

[6] R. Bellman, "Dynamic programming treatment of the travelling salesman problem," *J. ACM*, vol. 9, no. 1, pp. 61–63, 1962.

[7] J. D. C. Little, K. G. Murty, D. W. Sweeney, and C. Karel, "An algorithm for the traveling salesman problem," *Oper. Res.*, vol. 11, no. 6, pp. 972–989, 1963.

[8] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds. New York, NY, USA: Plenum, 1972, pp. 85–103.

[9] W. J. Cook, *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ, USA: Princeton Univ. Press, 2012.

[10] G. Reinelt, "TSPLIB—A traveling salesman problem library," *ORSA J. Comput.*, vol. 3, no. 4, pp. 376–384, 1991.

[11] A. R. Karlin, N. Klein, and S. Oveis Gharan, "A (slightly) improved approximation algorithm for metric TSP," in *Proc. 53rd Annu. ACM SIGACT Symp. Theory Comput. (STOC)*, Virtual Event, Jun. 2021, doi: 10.1145/3406325.3451009.

[12] O. Svensson, J. Tarnawski, and L. A. Végh, "A constant-factor approximation algorithm for the asymmetric traveling salesman problem," *J. ACM*, vol. 67, no. 6, 2020, doi: 10.1145/3424306.

[13] V. Traub and J. Vygen, *Approximation Algorithms for Traveling Salesman Problems*. Cambridge, U.K.: Cambridge Univ. Press, 2024.

[14] P. Toth and D. Vigo, Eds., *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA, USA: SIAM, 2014.

[15] N. Christofides, "Worst-case analysis of a new heuristic for the travelling salesman problem," Graduate School of Industrial Administration, Carnegie Mellon Univ., Pittsburgh, PA, USA, Rep. 388, 1976.

[16] A. I. Serdyukov, "On some extremal walks in graphs" (in Russian), *Upravlyaemye Sistemy*, vol. 17, pp. 76–79, 1978.

[17] S. Lin and B. W. Kernighan, "An effective heuristic algorithm for the traveling-salesman problem," *Oper. Res.*, vol. 21, no. 2, pp. 498–516, 1973.

[18] K. Helsgaun, "An effective implementation of the Lin–Kernighan traveling salesman heuristic," *Eur. J. Oper. Res.*, vol. 126, no. 1, pp. 106–130, Oct. 2000.

[19] S. Sahni and T. Gonzalez, "P-complete approximation problems," *J. ACM*, vol. 23, no. 3, pp. 555–565, Jul. 1976.

[20] S. Arora, "Polynomial time approximation schemes for Euclidean traveling salesman and other geometric problems," *J. ACM*, vol. 45, no. 5, pp. 753–782, Sep. 1998.

[21] M. Held and R. M. Karp, "The traveling-salesman problem and minimum spanning trees," *Oper. Res.*, vol. 18, no. 6, pp. 1138–1162, 1970.

[22] M. Held and R. M. Karp, "The traveling-salesman problem and minimum spanning trees: Part II," *Math. Program.*, vol. 1, no. 1, pp. 6–25, 1971.

[23] M. Karpinski, M. Lampis, and R. Schmied, "New inapproximability bounds for TSP," arXiv:1303.6437, 2013.

[24] D. P. Williamson and D. B. Shmoys, *The Design of Approximation Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2011.

[25] V. V. Vazirani, *Approximation Algorithms*. Berlin, Germany: Springer, 2001.

[26] D. S. Johnson and L. A. McGeoch, "Experimental analysis of heuristics for the STSP," in *The Traveling Salesman Problem and Its Variations*, G. Gutin and A. P. Punnen, Eds. Dordrecht, The Netherlands: Kluwer, 2002, pp. 369–443.

[27] G. A. Croes, "A method for solving traveling-salesman problems," *Oper. Res.*, vol. 6, no. 6, pp. 791–812, 1958.

[28] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis II, "An analysis of several heuristics for the traveling salesman problem," *SIAM J. Comput.*, vol. 6, no. 3, pp. 563–581, 1977.

[29] B. Chandra, H. Karloff, and C. Tovey, "New results on the old k-opt algorithm for the traveling salesman problem," *SIAM J. Comput.*, vol. 28, no. 6, pp. 1998–2029, 1999.

[30] S. Kirkpatrick, C. D. Gelatt, Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[31] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: Univ. Michigan Press, 1975.

[32] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.

[33] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 26, no. 1, pp. 29–41, Feb. 1996.

[34] M. Dorigo and L. M. Gambardella, "Ant colony system: A cooperative learning approach to the traveling salesman problem," *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 53–66, Apr. 1997.

[35] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[36] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, 2015.

[37] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," arXiv:1611.09940, 2016.

[38] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[39] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: A methodological tour d'horizon," *Eur. J. Oper. Res.*, vol. 290, no. 2, pp. 405–421, 2021.

[40] R. C. Prim, "Shortest connection networks and some generalizations," *Bell Syst. Tech. J.*, vol. 36, no. 6, pp. 1389–1401, 1957.

[41] J. Edmonds, "Paths, trees, and flowers," *Can. J. Math.*, vol. 17, pp. 449–467, 1965.

[42] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python Sci. Conf. (SciPy 2008)*, Pasadena, CA, USA, 2008, pp. 11–15.

[43] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020.

[44] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Manage. Sci.*, vol. 6, no. 1, pp. 80–91, 1959.

Appendix A: Additional Pseudocode

These algorithms are referred to in Chapters 4 and 5 and correspond to the functions of `tsp_survey.py`.

Algorithm A.1 HELD-KARP(D) (exact dynamic programming, $O(n^2 2^n)$ time)

```
let m = n − 1 and relabel cities 1..m so that city 0 is the start
f[{j}][j] ← D[0][j]  for every j
for every subset S ⊆ {1..m} in increasing order of size (|S| ≥ 2) do
   for every j ∈ S do
      f[S][j] ← min over i ∈ S\{j} of ( f[S\{j}][i] + D[i][j] )         remember the
      minimising i
cost ← min over j of ( f[{1..m}][j] + D[j][0] )
recover the tour by following the remembered predecessors from the best j back to city
0
```

Algorithm A.2 OR-OPT(D, tour) (segments of 1 to 3 cities)

```
repeat
   improved ← false
   for L ← 3 down to 1 do
      for each segment seg of L consecutive cities, with neighbours prev and next in
      the tour do
         removal gain ← D[prev][seg.first] + D[seg.last][next] − D[prev][next]
         for each edge (u, v) of the tour without seg do
            cost of inserting seg (in either orientation) ← min( D[u][first] + D[last]
            [v], D[u][last] + D[first][v] ) − D[u][v]
         if the best insertion cost < removal gain then move seg there;  improved ← true
until improved = false
```

Algorithm A.3 THREE-OPT (segment exchange, first improvement)

```
repeat until no improvement
   for i < j < k:  remove edges (a,b1)=(t[i],t[i+1]), (b2,c1)=(t[j],t[j+1]),
   (c2,d)=(t[k],t[k+1])
      let S1 = t[i+1..j] and S2 = t[j+1..k]
      evaluate the four reconnections that use three new edges:
         A S2 S1 D     A S2' S1 D     A S2 S1' D     A S1' S2' D        (' means
         reversed)
      if the cheapest reconnection is shorter than the removed edges then apply it
```

Algorithm A.4 EULER-AND-SHORTCUT(H) (Hierholzer, used in Christofides)

```
stack ← [0];  circuit ← []
while stack not empty do
   v ← top of stack
   if v has an unused edge (v, w) then remove that edge;  push w
   else pop v and append it to circuit
tour ← the cities of circuit in order, keeping only the first occurrence of each
```

Appendix B: Validation Code

The complete programme is supplied as the separate file `tsp_survey.py`. Only the tour-validation function, through which every reported tour passes, is reproduced here; it is the code that was run. Dependencies: Python 3, NumPy, NetworkX.

validate()

```
def validate(Dn, tour):
    """Return (is_valid_hamiltonian_cycle, independently recomputed cost)."""
    n = len(Dn)
    ok = len(tour) == n and sorted(tour) == list(range(n))
    if not ok:
        return False, float("nan")
    t = np.array(tour)
    return True, float(Dn[t, np.roll(t, -1)].sum())
```

Appendix C: Sample Cost Matrices and Test Cases

C.1 five-city illustration of Figure 2.1 (A,B,C,D,E; symmetric, NOT metric)

```
0, 12, 10, 19, 8
12, 0, 3, 7, 2
10, 3, 0, 6, 20
19, 7, 6, 0, 4
8, 2, 20, 4, 0
```

C.2 sample/demo_8city.csv (Manhattan distances; symmetric and metric)

```
0,8,8,16,10,13,12,4
8,0,6,8,12,5,6,4
8,6,0,8,6,7,4,4
16,8,8,0,8,5,4,12
10,12,6,8,0,13,6,10
13,5,7,5,13,0,7,9
12,6,4,4,6,7,0,8
4,4,4,12,10,9,8,0
```

C.3 sample/nonmetric_6city.csv (symmetric, triangle inequality violated)

```
0,9,18,24,19,81
9,0,4,10,34,44
18,4,0,16,70,74
24,10,16,0,89,52
19,34,70,89,0,74
81,44,74,52,74,0
```

C.4 sample/asym_5city.csv (asymmetric)

```
0,10,15,20,25
12,0,35,25,30
18,32,0,30,20
22,28,33,0,15
27,29,21,17,0
```

Table C.1: Test cases

#	Test	Expected behaviour	Result
T1	Brute force and Held–Karp on 30 instances of 8 cities (15 metric, 15 non-metric)	Equal optimal cost	Passed (self-test)
T2	Bitmask matching versus NetworkX matching on 20 instances	Equal matching weight	Passed (self-test)
T3	Every algorithm for n = 4, 5, 6, 7	Valid tour (each city once)	Passed (self-test)
T4	All heuristics against exact optimum on 10 instances of 9 cities	No heuristic cost below the optimum	Passed (self-test)
T5	1-tree lower bound against optimum on 10 instances of 10 cities	Bound ≤ optimum	Passed (self-test)
T6	Run on C.4 (asymmetric)	Symmetric-only methods skipped; tours valid	Observed: see Appendix D
T7	Run on C.3 (non-metric)	Warning that guarantees do not apply	Observed: see Appendix D
T8	Input with a non-zero diagonal, or with non-numeric entries	Clear error message, exit code 2	Observed in manual tests

Appendix D: Sample Program Outputs

Output of `python3 tsp_survey.py --matrix <file>` on the three sample files (one run each; run times vary between runs).

D.1 sample/demo_8city.csv

```
Loaded 8x8 matrix: symmetric=True, metric=True
Method                          Cost    Gap%   Time(ms)  Tour
Held-Karp DP (exact)           42.00    0.00      0.368  [0, 7, 2, 4, 6, 3, 5, 1, 0]
Brute Force (exact)            42.00    0.00      1.226  [0, 1, 5, 3, 6, 4, 2, 7, 0]
Nearest Neighbour              42.00    0.00      0.008  [0, 7, 1, 5, 3, 6, 2, 4, 0]
Cheapest Insertion             42.00    0.00      0.020  [0, 4, 2, 6, 3, 5, 1, 7, 0]
Farthest Insertion             42.00    0.00      0.024  [0, 7, 1, 5, 3, 6, 4, 2, 0]
Double-Tree                    52.00   23.81      0.044  [0, 7, 1, 5, 2, 4, 6, 3, 0]
Christofides                   42.00    0.00      0.062  [0, 7, 1, 5, 3, 6, 2, 4, 0]
2-opt (from NN)                42.00    0.00      0.012  [0, 7, 1, 5, 3, 6, 2, 4, 0]
Or-opt (from NN)               42.00    0.00      0.047  [0, 7, 1, 5, 3, 6, 2, 4, 0]
2-opt+Or-opt (from NN)         42.00    0.00      0.069  [0, 7, 1, 5, 3, 6, 2, 4, 0]
3-opt (from NN)                42.00    0.00      0.059  [0, 7, 1, 5, 3, 6, 2, 4, 0]
Simulated Annealing            42.00    0.00     61.464  [0, 7, 1, 5, 3, 6, 2, 4, 0]
Genetic Algorithm              42.00    0.00    118.962  [3, 5, 1, 0, 7, 2, 4, 6, 3]
Ant Colony (AS)                42.00    0.00     32.445  [0, 7, 1, 5, 3, 6, 2, 4, 0]
All tours validated: every city appears once; cost recomputed independently.
```

D.2 sample/nonmetric_6city.csv

```
Loaded 6x6 matrix: symmetric=True, metric=False
NOTE: triangle inequality violated -> the 2 (double-tree) and 3/2 (Christofides)
guarantees do NOT
Method                          Cost    Gap%   Time(ms)  Tour
Held-Karp DP (exact)          174.00    0.00      0.075  [0, 4, 5, 3, 2, 1, 0]
Brute Force (exact)           174.00    0.00      0.054  [0, 1, 2, 3, 5, 4, 0]
Nearest Neighbour             174.00    0.00      0.016  [0, 1, 2, 3, 5, 4, 0]
Cheapest Insertion            181.00    4.02      0.010  [0, 3, 2, 1, 5, 4, 0]
Farthest Insertion            174.00    0.00      0.009  [0, 1, 2, 3, 5, 4, 0]
Double-Tree                   174.00    0.00      0.019  [0, 1, 2, 3, 5, 4, 0]
Christofides                  174.00    0.00      0.030  [0, 1, 2, 3, 5, 4, 0]
2-opt (from NN)               174.00    0.00      0.006  [0, 1, 2, 3, 5, 4, 0]
Or-opt (from NN)              174.00    0.00      0.022  [0, 1, 2, 3, 5, 4, 0]
2-opt+Or-opt (from NN)        174.00    0.00      0.033  [0, 1, 2, 3, 5, 4, 0]
3-opt (from NN)               174.00    0.00      0.020  [0, 1, 2, 3, 5, 4, 0]
Simulated Annealing           174.00    0.00     43.331  [0, 1, 2, 3, 5, 4, 0]
Genetic Algorithm             174.00    0.00    107.553  [4, 0, 1, 2, 3, 5, 4]
Ant Colony (AS)               174.00    0.00     16.379  [3, 2, 1, 0, 4, 5, 3]
All tours validated: every city appears once; cost recomputed independently.
```

D.3 sample/asym_5city.csv

```
Loaded 5x5 matrix: symmetric=False, metric=False
Asymmetric matrix: MST-based methods and symmetric-cost local search/SA are skipped.
Method                          Cost    Gap%   Time(ms)  Tour
Held-Karp DP (exact)           89.00    0.00      0.038  [0, 1, 3, 4, 2, 0]
Nearest Neighbour              89.00    0.00      0.004  [0, 1, 3, 4, 2, 0]
Cheapest Insertion             89.00    0.00      0.007  [0, 1, 3, 4, 2, 0]
Farthest Insertion             92.00    3.37      0.007  [0, 2, 4, 3, 1, 0]
Genetic Algorithm              89.00    0.00    130.065  [2, 0, 1, 3, 4, 2]
Ant Colony (AS)                89.00    0.00     21.665  [2, 0, 1, 3, 4, 2]
All tours validated: every city appears once; cost recomputed independently.
```

Appendix E: Software Testing Checklist

Table E.1: Software testing checklist

Item	Status
Every tour returned by every algorithm is a permutation of all cities (validated in code)	√ Automated
Every reported cost is recomputed independently from the cost matrix	√ Automated
Optimal costs from brute force and Held–Karp agree (T1)	√ Automated
No heuristic returns a cost below the exact optimum (T4); lower bound never exceeds the optimum (T5)	√ Automated
Invalid input (non-zero diagonal, non-numeric entries, fewer than 4 cities) is rejected with an error message (T8)	√ Manual test
All numbers in Chapter 7 are taken from `results.json`, written by the programme	√ Script-generated
Fixed random seeds, so that costs are repeatable (run times vary between runs)	√